CS353 Design Report

24.03.2026

Group 18

Berra Alkış - 22302527 – Section 2

Doğa Ertürk - 22202567 – Section 2

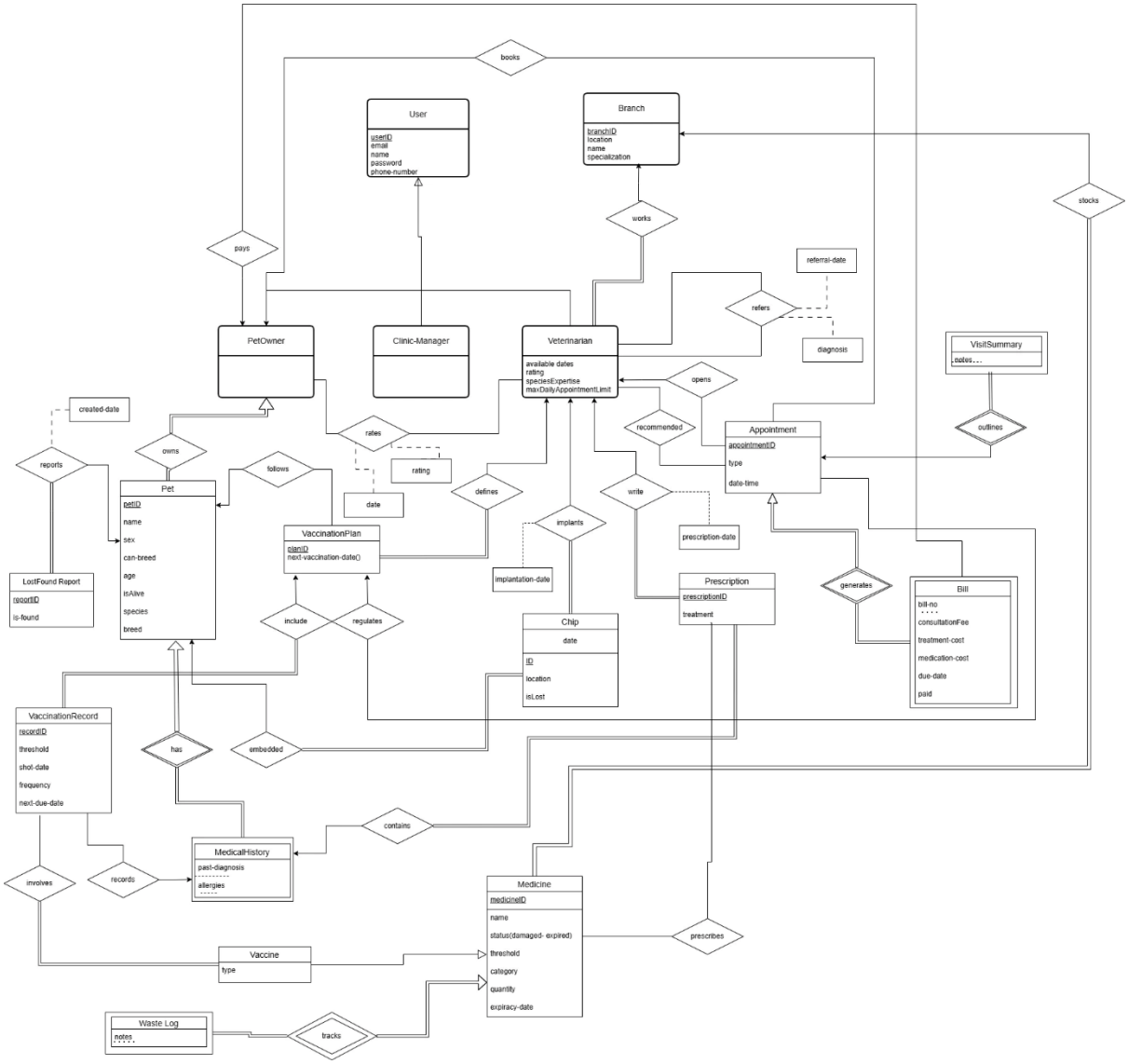
Yasemin Altun – 22202739 – Section 1

Melike Feyza Bahadırlı - 22201729 – Section 1

Ayşe Tayyibe Zehra Özkan - 22301879 – Section 1

Table of Contents

1. Revised E/R Diagram



For detailed information regarding the E/R diagram link here.

Revised Parts:

- Report, Inventory Report and Vaccination Report entities are removed for simplicity of the project.
- Supplier and Restock request along with their workflows have been removed for project simplicity.
- Pet-owns-Pet Owner relation set has been changed into one-to-many to restrick one pet having more than one owners.
- Bills entity has been changed to a weak entity and that depends on appointment entity with a generates relation.
- Referral Record entity has been removed and a new connection between the veterinarian and itself has been established through refers relationship.

- Shipment and inventoryItem entity have been removed from the e/r diagram for simplicity and vaccine has been made a type of medicine.
- WasteLog entity has been changed to a weak entity as its existence depended entirely on the medicine entity.
- Email attribute has been added to the User entity for easier identification.

2. Table Schemas

This section contains the abstract table schemas corresponding to the E/R diagram given in section 1 with underlined attributes referring to the primary key of the tables.

2.1. User Related Schemas

Branch(branchID, location, name, specialization)
Candiate Keys: {branchID}, {location}

```
CREATE TABLE Branch (
  branchID INT PRIMARY KEY,
  location VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,
  specialization VARCHAR(255)
);
```

User (userID, name, password, phoneNumber, email)
Candidate Keys: {userID}, {phoneNumber}, {email}

```
CREATE TABLE User (
  userID INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  password VARCHAR(255) NOT NULL,
  phoneNumber VARCHAR(20) UNIQUE NOT NULL,
 email VARCHAR(20) UNIQUE NOT NULL
);
```

PetOwner (ownerID)
Candidate key : {ownerID}
ownerID FK to User.userID

```
CREATE TABLE PetOwner (
        ownerID INT PRIMARY KEY,
```

FOREIGN KEY (ownerID) REFERENCES User(userID) ON DELETE CASCADE
);

ClinicManager(managerID)
Candidate key : {managerID}
managerID FK to User.userID

CREATE TABLE ClinicManager (
managerID INT PRIMARY KEY,
FOREIGN KEY (managerID) REFERENCES User(userID) ON DELETE CASCADE
);

Veterinarian (veterinarianID, availableDates, rating,
maxDailyAppointmentLimit,speciesExpertise, branchID)
Candidate key: {veterinarianID}
veterinarianID FK to User.userID
branchID FK to Branch.branchID for “works” relation

CREATE TABLE Veterinarian (
veterinarianID INT PRIMARY KEY,
availableDates VARCHAR(255),
rating DECIMAL(3, 2),
maxDailyAppointmentLimit INT,
speciesExpertise VARCHAR(255),
branchID INT,
FOREIGN KEY (veterinarianID) REFERENCES User(userID) ON DELETE CASCADE,
FOREIGN KEY (branchID) REFERENCES Branch(branchID) ON DELETE SET NULL
);

Rates (ownerID, veterinarianID, date, rating)
Candidate key: {ownerID, veterinarianID, date}
ownerID FK to PetOwner.ownerID
veterinarianID FK to Veterinarian.veterinarianID

CREATE TABLE Rates (
ownerID INT NOT NULL,
veterinarianID INT NOT NULL,
date DATE NOT NULL,

rating INT CHECK (rating BETWEEN 0 AND 10),
PRIMARY KEY (ownerID, veterinarianID, date),
FOREIGN KEY (ownerID) REFERENCES PetOwner(ownerID) ON DELETE CASCADE,
FOREIGN KEY (veterinarianID) REFERENCES Veterinarian(veterinarianID) ON DELETE CASCADE
);

Refers (referrer, referee, referralDate, diagnosis)
Candidate key: {referrer, referee, referralDate, diagnosis}
referrer FK to Veterinarian.veterinarianID
referee FK to Veterinarian.veterinarianID

CREATE TABLE Refers (
referrer INT NOT NULL,
referee INT NOT NULL,
referralDate DATE NOT NULL,
diagnosis TEXT,
PRIMARY KEY (referrer, referee, referralDate, diagnosis),
FOREIGN KEY (referrer) REFERENCES Veterinarian(veterinarianID) ON DELETE CASCADE,
FOREIGN KEY (referee) REFERENCES Veterinarian(veterinarianID) ON DELETE CASCADE
);

2.2. Pet Related Schemas

Pet (petID, name, sex, canBreed, age, isAlive, species, breed, ownerID)
Candidate key: {petID}
ownerID FK to PetOwner.ownerID for “owns” relation

CREATE TABLE Pet (
petID INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
sex ENUM(‘F’, ‘M’) NOT NULL DEFAULT ‘F’,
canBreed BOOLEAN,
age INT,
isAlive BOOLEAN,
species VARCHAR(100),
breed VARCHAR(100),

```
  ownerID INT NOT NULL,
  FOREIGN KEY (ownerID) REFERENCES PetOwner(ownerID) ON DELETE CASCADE
);
```

LostFoundReport(reportID, isFound, petID, createdDate)

Candiate key: {reportID}
petID FK to Pet.petID for “reports” relation

```
CREATE TABLE LostFoundReport (
  reportID INT PRIMARY KEY,
  isFound BOOLEAN DEFAULT TRUE,
  petID INT NOT NULL,
  createdDate DATE NOT NULL,
  FOREIGN KEY (petID) REFERENCES Pet(petID) ON DELETE CASCADE
);
```

Chip(chipID, location, isLost, petID, veterinarianID, implantationDate)
Candidate key: {chipID}
petID FK to Pet.petID for “embedded” relation
veterinarianID FK to Veterinarian.veterinarianID for “implants” relation

```
CREATE TABLE Chip (
  chipID INT PRIMARY KEY,
  location VARCHAR(255),
  isLost BOOLEAN DEFAULT FALSE,
  petID INT NOT NULL,
  veterinarianID INT NOT NULL DEFAULT -1,
  implantationDate DATE,
  FOREIGN KEY (petID) REFERENCES Pet(petID) ON DELETE CASCADE,
  FOREIGN KEY (veterinarianID) REFERENCES Veterinarian(veterinarianID) ON DELETE SET
DEFAULT
);
```

MedicalHistory(petID, pastDiagnosis, allergies)
Candidate key: { petID, pastDiagnosis, allergies}
petID FK to Pet.petID for “has” relation

```
CREATE TABLE MedicalHistory (
```

petID INT NOT NULL,
pastDiagnosis TEXT,
allergies TEXT,
PRIMARY KEY (petID, pastDiagnosis, allergies),
FOREIGN KEY (petID) REFERENCES Pet(petID) ON DELETE CASCADE
);

2.3. Appointment Related Schemas

VaccinationPlan(planID, nextVaccinationDate, petID, veterinarianID)
Candidate Key: {planID}
petID FK to Pet.petID for "follows" relation
veterinarianID FK to Veterinarian.veterinarianID for "defines" relation

CREATE TABLE VaccinationPlan (
planID INT PRIMARY KEY,
nextVaccinationDate DATE,
petID INT NOT NULL,
veterinarianID INT NOT NULL DEFAULT -1,
FOREIGN KEY (petID) REFERENCES Pet(petID) ON DELETE CASCADE,
FOREIGN KEY (veterinarianID) REFERENCES Veterinarian(veterinarianID) ON DELETE SET DEFAULT
);

Appointment(appointmentID, type, dateTime, vaccinationPlanID, veterinarianID, petOwnerID)
Candidate Key: {appointmentID}
petOwnerID FK to PetOwner.ownerID for "books" relation
vaccinationPlanID FK to VaccinationPlan.planID for "regulates" relation
veterinarianID FK to Veterinarian.veterinarianID for "opens" relation

CREATE TABLE Appointment (
appointmentID INT PRIMARY KEY,
type ENUM('CHECKUP', 'VACCINATION, 'COMPLAINT', 'EMERGENCY') NOT NULL DEFAULT 'CHECKUP',
dateTime DATETIME NOT NULL,
vaccinationPlanID INT,
veterinarianID INT NOT NULL,
petOwnerID INT NOT NULL,
FOREIGN KEY (vaccinationPlanID) REFERENCES VaccinationPlan(planID) ON DELETE SET NULL,

FOREIGN KEY (veterinarianID) REFERENCES Veterinarian(veterinarianID) ON DELETE CASCADE,
FOREIGN KEY (petOwnerID) REFERENCES PetOwner(ownerID) ON DELETE CASCADE
);

Recommended(veterinarianID, appointmentID)
Candidate key: {veterinarianID, appointmentID)
veterinarianID FK to Veterinarian.veterinarianID
appointmentID FK to Appointment.appointmentID

CREATE TABLE Recommended (
veterinarianID INT NOT NULL,
appointmentID INT NOT NULL,
PRIMARY KEY (veterinarianID, appointmentID),
FOREIGN KEY (veterinarianID) REFERENCES Veterinarian(veterinarianID) ON DELETE CASCADE,
FOREIGN KEY (appointmentID) REFERENCES Appointment(appointmentID) ON DELETE CASCADE
);

VisitSummary(appointmentID, notes)
Candiate Key: {appointmentID, notes}
appointmentID FK to Appointment.appointmentID for “outlines” relation

CREATE TABLE VisitSummary (
appointmentID INT NOT NULL,
notes TEXT NOT NULL,
PRIMARY KEY (appointmentID, notes),
FOREIGN KEY (appointmentID) REFERENCES Appointment(appointmentID) ON DELETE CASCADE
);

Bill(billNo, appointmentID, consultationFee, treatmentCost, medicationCost, dueDate, paid, payerID)
Candiadate key: {billNo, appointmentID}
appointmentID FK to Appointment.appointmentID for “generates” relation
payerID FK to PetOwner.ownerID for “pays” relation

CREATE TABLE Bill (
billNo INT NOT NULL,

appointmentID INT NOT NULL,
consultationFee DECIMAL(10, 2) DEFAULT 0,
treatmentCost DECIMAL(10, 2) DEFAULT 0,
medicationCost DECIMAL(10, 2) DEFAULT 0,
dueDate DATE,
paid BOOLEAN DEFAULT FALSE,
payerID INT NOT NULL,
PRIMARY KEY (billNo, appointmentID),
FOREIGN KEY (appointmentID) REFERENCES Appointment(appointmentID) ON DELETE CASCADE,
FOREIGN KEY (payerID) REFERENCES PetOwner(ownerID) ON DELETE CASCADE
);

2.4. Medicine Related Schemas

Medicine(medicineID, name, status, threshold, category, quantity, expiracyDate, branchID)
Candidate key: {medicineID}
branchID FK to Branch.branchID for “stocks” relation

CREATE TABLE Medicine (
medicineID INT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
status ENUM('damaged', 'safe', 'expired') DEFAULT 'safe',
threshold INT,
category ENUM(‘A’, ‘B’, ‘C’, ‘D’),
quantity INT,
expiracyDate DATE,
branchID INT NOT NULL,
FOREIGN KEY (branchID) REFERENCES Branch(branchID) ON DELETE CASCADE
);

WasteLog(medicineID, notes)
Candidate key: {medicineID, notes}
medicineID FK to Medicine.medicineID for “tracks” relation

CREATE TABLE WasteLog (
medicineID INT NOT NULL,
notes TEXT NOT NULL,
PRIMARY KEY (medicineID, notes),

FOREIGN KEY (medicineID) REFERENCES Medicine(medicineID) ON DELETE CASCADE
);

Vaccine(vaccineID, type)
Candidate key: {vaccineID}
vaccineID FK to Medicine.medicineID

CREATE TABLE Vaccine (
vaccineID INT PRIMARY KEY,
type VARCHAR(100),
FOREIGN KEY (vaccineID) REFERENCES Medicine(medicineID) ON DELETE CASCADE
);

VaccinationRecord(recordID, threshold, shotDate, frequency, nextDueDate, planID, petID, pastDiagnosis, allergies)
Candidate key: {recordID}
planID FK to VaccinationPlan.planID for “include” relation
(petID, pastDiagnosis, allergies) FK to (MedicalHistory.petID, MedicalHistory.pastDiagnosis, MedicalHistory.allergies) for “records” relation

CREATE TABLE VaccinationRecord (
recordID INT PRIMARY KEY,
threshold INT,
shotDate DATE,
frequency VARCHAR(50),
nextDueDate DATE,
planID INT NOT NULL,
petID INT NOT NULL,
pastDiagnosis TEXT,
allergies TEXT,
FOREIGN KEY (planID) REFERENCES VaccinationPlan(planID) ON DELETE CASCADE,
FOREIGN KEY (petID, pastDiagnosis, allergies) REFERENCES MedicalHistory(petID, pastDiagnosis, allergies) ON DELETE CASCADE
);

Involves(recordID, vaccineID)
Candidate key: {recordID, vaccineID}
recordID FK to VaccinationRecord.recordID
vaccineID FK to Vaccine.vaccineID

CREATE TABLE Involves (
recordID INT NOT NULL,
vaccineID INT NOT NULL,
PRIMARY KEY (recordID, vaccineID),
FOREIGN KEY (recordID) REFERENCES VaccinationRecord(recordID) ON DELETE CASCADE,
FOREIGN KEY (vaccineID) REFERENCES Vaccine(vaccineID) ON DELETE CASCADE
);

Prescription(prescriptionID, treatment, veterinarianID, petID, pastDiagnosis, allergies, prescriptionDate)
Candidate key: {prescriptionID}
veterinatianID FK to Veterinarian.veterinarianID for "writes" relation
(petID, pastDiagnosis, allergies) FK to (MedicalHistory.petID, MedicalHistory.pastDiagnosis, MedicalHistory.allergies) for "contains" relation

CREATE TABLE Prescription (
prescriptionID INT PRIMARY KEY,
treatment TEXT,
veterinarianID INT NOT NULL DEFAULT -1,
petID INT NOT NULL,
pastDiagnosis TEXT,
allergies TEXT,
prescriptionDate DATE,
FOREIGN KEY (veterinarianID) REFERENCES Veterinarian(veterinarianID) ON DELETE SET DEFAULT,
FOREIGN KEY (petID, pastDiagnosis, allergies) REFERENCES MedicalHistory(petID, pastDiagnosis, allergies) ON DELETE CASCADE
);

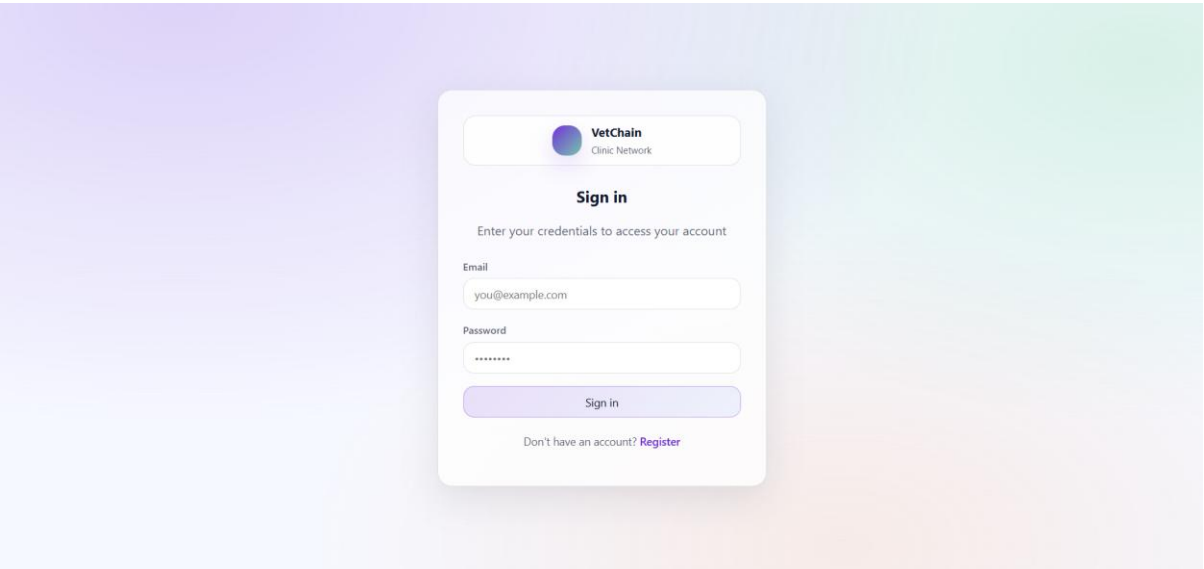
Prescribes(prescriptionID , medicineID)
Candidate key: {prescriptionID , medicineID}
prescriptionID FK to Prescription.prescriptionID
veterinarianID FK to Veterinarian.veterinarianID

CREATE TABLE Prescribes (
prescriptionID INT NOT NULL,
medicineID INT NOT NULL,
PRIMARY KEY (prescriptionID, medicineID),

FOREIGN KEY (prescriptionID) REFERENCES Prescription(prescriptionID) ON DELETE CASCADE,
FOREIGN KEY (medicineID) REFERENCES Medicine(medicineID) ON DELETE CASCADE
);

3. User Interface Design

3.1.1 Login Page

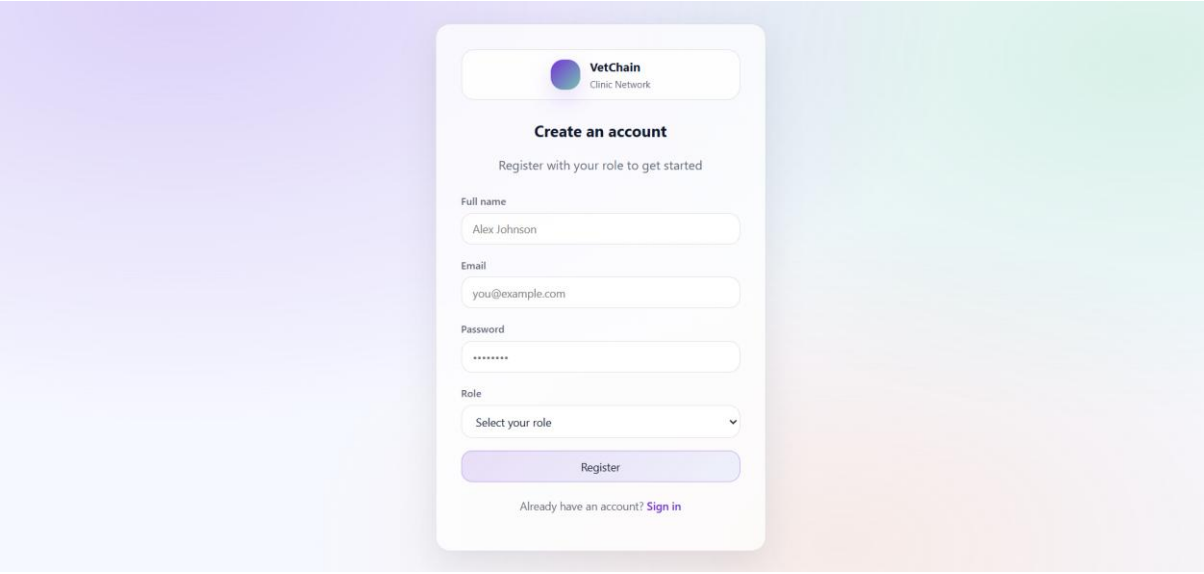


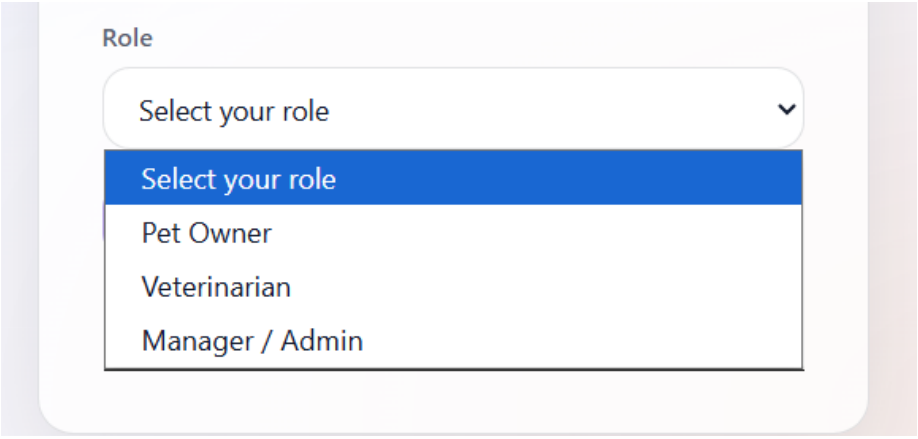
The login page allows an existing user to access the system by entering their e-mail and password. Since role information is not stored directly in the User table, the system authenticates the user first and then determines whether the user is a PetOwner, Veterinarian, or ClinicManager by checking the corresponding role tables. This is necessary because the system provides role-based access to different interfaces.

SELECT

```
  u.userID,
  u.name,
  u.email,
  CASE
    WHEN po.ownerID IS NOT NULL THEN 'PetOwner'
    WHEN v.veterinarianID IS NOT NULL THEN 'Veterinarian'
    WHEN cm.managerID IS NOT NULL THEN 'ClinicManager'
    ELSE 'Unknown'
  END AS userRole
FROM User u
LEFT JOIN PetOwner po ON po.ownerID = u.userID
LEFT JOIN Veterinarian v ON v.veterinarianID = u.userID
LEFT JOIN ClinicManager cm ON cm.managerID = u.userID
WHERE u.email = ? AND u.password = ?;
```

3.1.2 Register Page with Role Specification





This page allows a new user to create an account by entering full name, e-mail address, phone number, password, and selecting a role. The system first checks whether the entered e-mail address or phone number already exists. If the information is unique, the user is inserted into the User table. Then, depending on the selected role, the same userID is inserted into PetOwner, Veterinarian, or ClinicManager. This design is consistent with the role separation in the database schema.

First, the system checks for duplicate e-mail or phone number before creating the account:

```
SELECT userID, name, email, phoneNumber
FROM User
WHERE email = 'doga.erturk@ug.bilkent.edu.tr'
  OR phoneNumber = '0532-410-1122';
```

Insert the new user into the User table

Example for a pet owner registration:

```
INSERT INTO User (userID, name, password, phoneNumber, email)
VALUES (101, 'Doğa Ertürk', 'doga123vet', '0532-410-1122', 'doga.erturk@ug.bilkent.edu.tr');
```

Example for a clinic manager registration:

```
INSERT INTO User (userID, name, password, phoneNumber, email)
VALUES (102, 'Berra Alkış', 'berraClinic2026', '0533-520-3344', 'berra.alkis@ug.bilkent.edu.tr');
```

Example for a veterinarian registration:

```
INSERT INTO User (userID, name, password, phoneNumber, email)
VALUES (103, 'Yasemin Altun', 'yaseminVet456', '0535-612-7788',
'yasemin.altun@ug.bilkent.edu.tr');
```

Then, continuing with registration with roles:

If the selected role is Pet Owner

```
INSERT INTO PetOwner (ownerID)
VALUES (101);
```

If the selected role is Clinic Manager

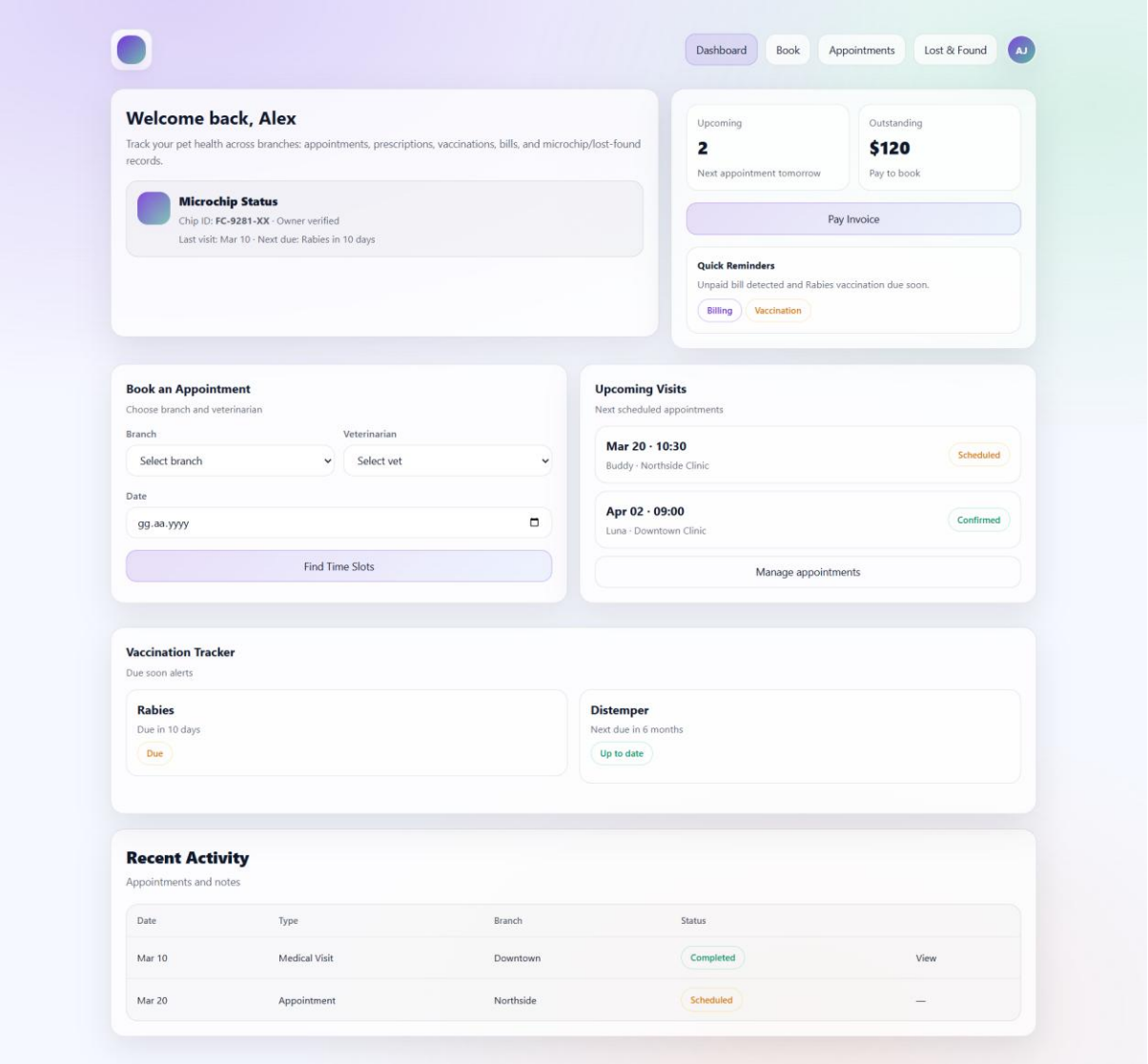
```
INSERT INTO ClinicManager (managerID)
VALUES (102);
```

If the selected role is Veterinarian

```
INSERT INTO Veterinarian
(veterinarianID, availableDates, rating, maxDailyAppointmentLimit, speciesExpertise, branchID)
VALUES
(103, 'Mon-Wed-Fri 09:00-17:00', NULL, 8, 'Cats, Small Dogs', 1);
```

3.2 Pages Accessible by Pet Owner

3.2.1 Pet Owner Dashboard



This page serves as the main overview screen for pet owners after login. It summarizes the owner's upcoming appointments, billing status, vaccination reminders, recent activity, and microchip/lost-found related information. The dashboard mainly uses SELECT queries to retrieve personalized data belonging to the currently logged-in pet owner. Since the system is role-based, all records shown on this page are filtered by the logged-in owner's ID. The page may also allow the owner to start a booking operation or update lost/found related status through additional actions.

Welcome Information

This section shows the basic account information of the currently logged-in pet owner.

```
SELECT u.userID, u.name, u.email, u.phoneNumber
FROM User u
JOIN PetOwner po ON po.ownerID = u.userID
WHERE po.ownerID = 101;
```

Upcoming Appointments Count

```
SELECT COUNT(*) AS upcomingAppointmentCount
FROM Appointment
WHERE petOwnerID = 101
  AND dateTime > '2026-03-24 00:00:00';
```

This query counts future appointments of the logged-in owner by filtering on petOwnerID and appointment date. It is used for the small summary card at the top-right of the dashboard.

Outstanding Bills Summary

```
SELECT
  SUM(consultationFee + treatmentCost + medicationCost) AS outstandingAmount
FROM Bill
WHERE payerID = 101
  AND paid = FALSE;
```

This query calculates the owner's unpaid balance by summing consultation, treatment, and medication costs from unpaid bills. It supports the "Outstanding" or "Pay to book" style warning shown on the dashboard. Bill is directly linked to the pet owner through payerID.

Microchip Status

```
SELECT
  p.petID,
   p.name AS petName,
   c.chipID,
   c.location,
   c.isLost,
   c.implantationDate,
   u.name AS veterinarianName
FROM Pet p
JOIN Chip c ON c.petID = p.petID
LEFT JOIN Veterinarian v ON v.veterinarianID = c.veterinarianID
LEFT JOIN User u ON u.userID = v.veterinarianID
WHERE p.ownerID = 101;
```

This query lists the pet's chip information, implantation date, chip location, and the veterinarian who implanted the chip. It is suitable for the microchip status widget shown on the left side of the dashboard. Since Chip is connected to Pet, and Pet is connected to PetOwner, filtering by owner makes the result personalized.

Upcoming Visits List

```
SELECT
  a.appointmentID,
   a.dateTime,
   a.type,
```

```
    uv.name AS veterinarianName,
    b.name AS branchName,
    b.location
FROM Appointment a
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
JOIN User uv ON uv.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE a.petOwnerID = 101
  AND a.dateTime >= '2026-03-24 00:00:00'
ORDER BY a.dateTime ASC;
```

This query is used to fill the “Upcoming Visits” panel. It combines appointment details with veterinarian and branch information so that the owner can clearly see where and with whom the appointment is scheduled.

Recent Activity

```
SELECT
    a.dateTime AS activityDate,
    'Appointment' AS activityType,
    a.type AS detail,
    b.name AS branchName
FROM Appointment a
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE a.petOwnerID = 101

UNION

SELECT
    CAST(bl.dueDate AS DATETIME) AS activityDate,
    'Bill' AS activityType,
    CONCAT('Bill #', bl.billNo) AS detail,
    NULL AS branchName
FROM Bill bl
WHERE bl.payerID = 101
ORDER BY activityDate DESC;
```

This query merges recent appointment actions and bill-related activities into a single timeline-like result. It is appropriate for the “Recent Activity” table shown at the bottom of the dashboard.

Find Time Slots Button / Start Booking Action

The booking card on the dashboard may allow the owner to search for available veterinarians by branch and date. Since your schema stores veterinarian availability in availableDates, a simple filtering query can be used.

```
SELECT
  v.veterinarianID,
   u.name AS veterinarianName,
   v.speciesExpertise,
   v.availableDates,
   b.name AS branchName
FROM Veterinarian v
JOIN User u ON u.userID = v.veterinarianID
JOIN Branch b ON b.branchID = v.branchID
WHERE b.branchID = 1
 AND v.availableDates LIKE '%2026-03-30%';
```

This query can be triggered when the owner selects a branch and date and clicks “Find Time Slots”. It is a retrieval query used to populate the booking widget with matching veterinarians.

Mark a Pet as Lost / Update Chip Status

If the owner creates a lost report from the dashboard, an update/insert operation can also be shown since the section should include both retrieval and modification queries.

Update chip status:

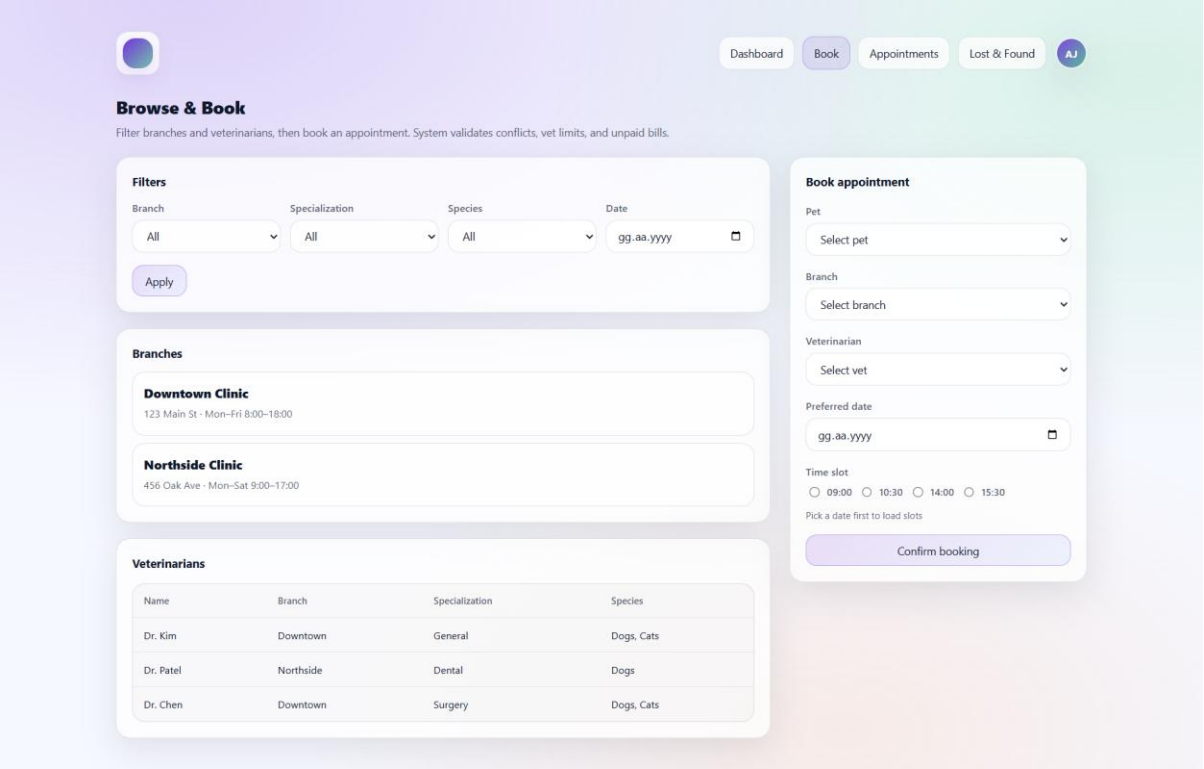
```
UPDATE Chip
SET isLost = TRUE
WHERE chipID = 88001234
 AND petID IN (
    SELECT petID
    FROM Pet
    WHERE ownerID = 101
 );
```

Create a lost/found report:

```
INSERT INTO LostFoundReport (reportID, isFound, petID)
VALUES (301, FALSE, 205);
```

These queries support the microchip and lost/found functionality mentioned in the project proposal. The first one marks the chip as lost, while the second one creates a lost report for the owner’s pet.

3.2.2 Browse Branches and Book Appointment



This page allows pet owners to browse available branches and veterinarians, apply filters such as branch, specialization, species expertise, and date, and then create a new appointment. According to the project functionality definition, this page must support both browsing and validated booking. Therefore, the interface uses SELECT queries to retrieve suitable branches, veterinarians, pets, and existing appointments, and it uses an INSERT query to create a new appointment after validation.

Load available branches

When the page is opened, the system first retrieves the branch list so that the user can browse available clinic locations.

```
SELECT branchID, name, location, specialization
FROM Branch
ORDER BY name;
```

Browse veterinarians with filters

```
SELECT
  v.veterinarianID,
   u.name AS veterinarianName,
   b.name AS branchName,
   v.speciesExpertise,
   v.availableDates,
```

```
  v.rating
FROM Veterinarian v
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE b.branchID = 1
 AND v.speciesExpertise LIKE '%Cats%'
ORDER BY u.name;
```

This query is used for the “Veterinarians” table shown on the page. It helps the user browse suitable veterinarians based on selected branch and species expertise. The veterinarian’s display name comes from User, while branch and expertise information come from Veterinarian and Branch.

Check unpaid bills before booking

According to the project requirements, booking should be blocked if the owner has unpaid bills. Therefore, the system must first check whether the owner has outstanding payments.

```
SELECT billNo, dueDate, consultationFee, treatmentCost, medicationCost
FROM Bill
WHERE payerID = 101
 AND paid = FALSE;
```

If this query returns any row, the booking request is rejected and the owner is informed that unpaid bills must be cleared first.

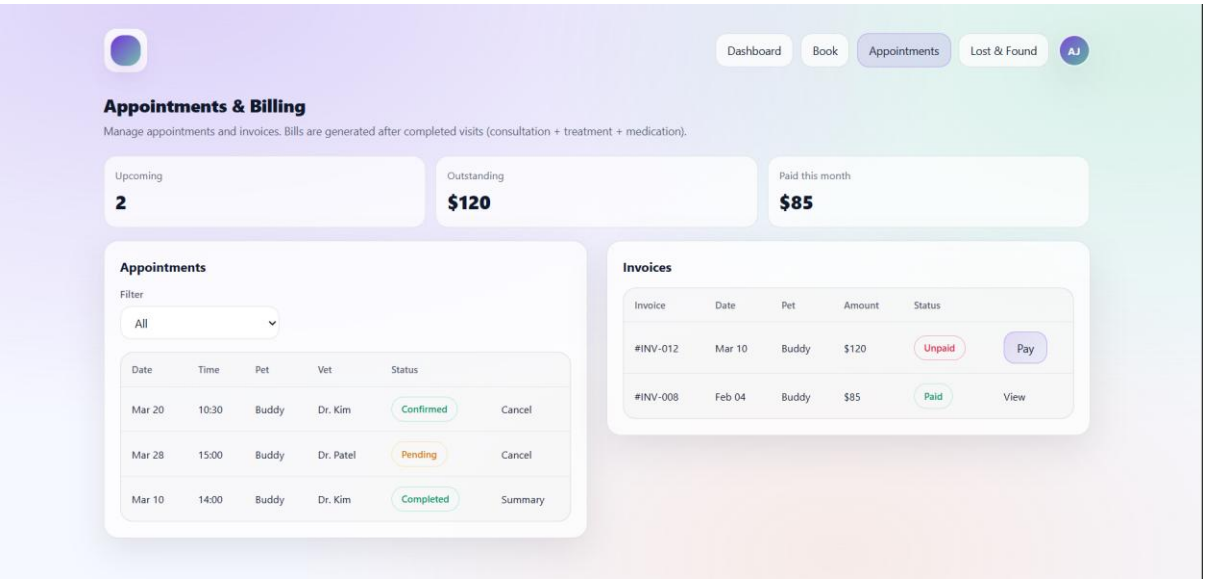
Create the appointment

If the selected time slot is available, the owner has no unpaid bills, and the veterinarian has not exceeded the daily limit, the system creates a new appointment.

```
INSERT INTO Appointment
(appointmentID, type, dateTime, vaccinationPlanID, veterinarianID, petOwnerID)
VALUES
(401, 'CHECKUP', '2026-03-30 10:30:00', NULL, 103, 101);
```

This query inserts the new booking into the Appointment table. The appointment is associated with exactly one veterinarian and one pet owner, which matches the current schema design. Since vaccinationPlanID is optional, it can be left NULL for general checkup appointments.

3.2.3 View Appointments and Billing



This page allows pet owners to view their appointment records and billing information in one place. The page combines appointment-related data with invoice details so that users can monitor their upcoming visits, completed visits, unpaid bills, and previously paid bills. In the current schema, appointment information is stored in the Appointment table, while billing information is stored in the Bill table.

Load appointment list of the pet owner

```
SELECT
  a.appointmentID,
   a.type,
   a.dateTime,
   u.name AS veterinarianName,
   b.name AS branchName
FROM Appointment a
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE a.petOwnerID = 101
ORDER BY a.dateTime DESC;
```

This query is used to populate the appointments table on the left side of the page. Since the current schema does not include an appointment status attribute, the interface primarily shows appointment type, date-time, veterinarian, and branch information.

Load invoice list of the pet owner

This query retrieves billing records of the pet owner together with the related appointment information.

```
SELECT
  bl.billNo,
   bl.appointmentID,
   bl.consultationFee,
   bl.treatmentCost,
   bl.medicationCost,
   bl.dueDate,
   bl.paid
FROM Bill bl
WHERE bl.payerID = 101
ORDER BY bl.dueDate DESC;
```

This query fills the invoice table on the right side of the page. It allows the user to see all generated bills and whether they are already paid or still unpaid. The bill status shown in the interface can be directly derived from the paid attribute.

Load only unpaid invoices

```
SELECT
  billNo,
   appointmentID,
   consultationFee,
   treatmentCost,
   medicationCost,
   dueDate
FROM Bill
WHERE payerID = 101
 AND paid = FALSE
ORDER BY dueDate ASC;
```

This result can be used for the “Outstanding” section at the top of the page. It helps the user identify unpaid invoices that may block new appointments.

Calculate total outstanding amount

```
SELECT
  SUM(consultationFee + treatmentCost + medicationCost) AS outstandingAmount
FROM Bill
WHERE payerID = 101
 AND paid = FALSE;
```

This query is suitable for the summary card such as “Outstanding: $120” displayed at the top of the page.

Calculate amount paid this month

```
SELECT
  SUM(consultationFee + treatmentCost + medicationCost) AS paidThisMonth
FROM Bill
WHERE payerID = 101
 AND paid = TRUE
 AND dueDate BETWEEN '2026-03-01' AND '2026-03-31';
```

This query supports the “Paid this month” summary box shown in the interface.

Mark an invoice as paid

This page may also support bill payment. In that case, the following update query can be used.

```
UPDATE Bill
SET paid = TRUE
WHERE billNo = 12
 AND appointmentID = 401
 AND payerID = 101;
```

This query updates the selected invoice after the user completes a payment action. It is a data modification operation that changes the bill status from unpaid to paid.

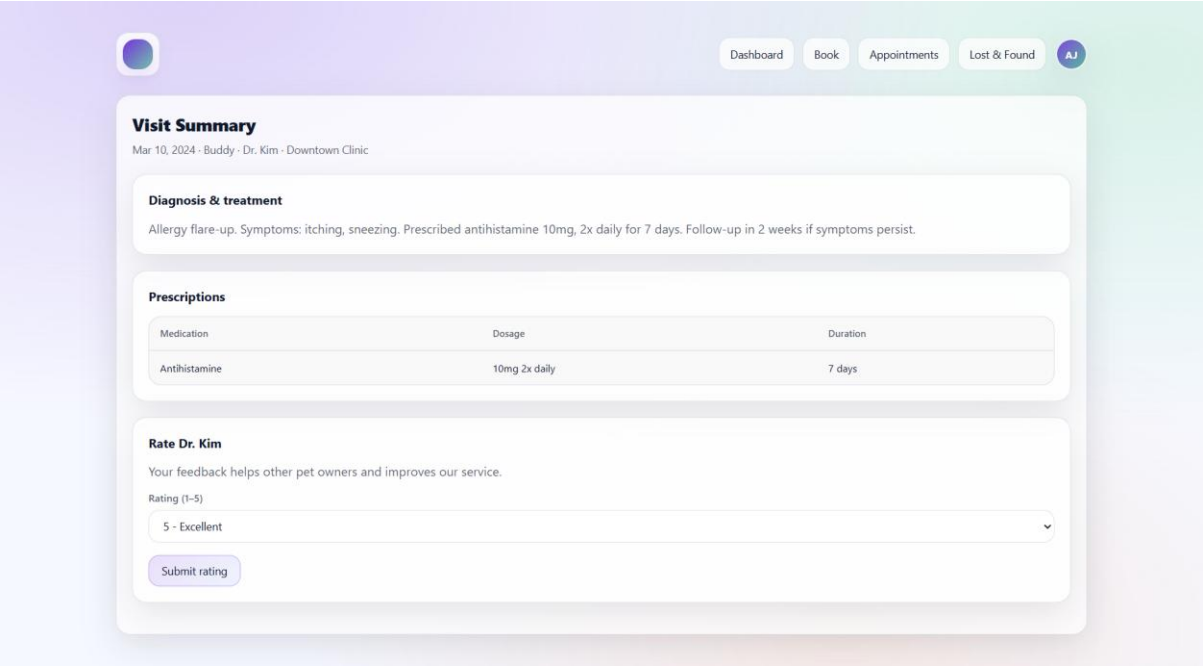
Cancel an upcoming appointment

If cancellation is supported by the interface, the following delete operation can be used for an appointment that belongs to the current owner.

```
DELETE FROM Appointment
WHERE appointmentID = 401
 AND petOwnerID = 101
 AND dateTime > '2026-03-24 00:00:00';
```

This query removes a future appointment that the owner wants to cancel. It is included as a modification query to show that the page supports not only viewing records but also managing appointments.

3.2.4 View Visit Summary



This page allows the pet owner to view the summary of a completed visit, inspect prescribed medication, and submit a rating for the veterinarian. In the current schema, visit details are represented by the VisitSummary table, prescriptions are stored in Prescription and Prescribes, and veterinarian evaluations are stored in the Rates table. Therefore, this page uses SELECT queries to display visit-related information and an INSERT query to record the pet owner's rating.

Load visit summary notes

```
SELECT
  vs.appointmentID,
   vs.notes
FROM VisitSummary vs
WHERE vs.appointmentID = 401;
```

This query is used to populate the "Diagnosis & treatment" section of the page. Since the current schema stores summary text in the VisitSummary table, the diagnosis and treatment explanation shown in the interface is derived from the notes attribute.

Load veterinarian and appointment information

```
SELECT
  a.appointmentID,
   a.dateTime,
   a.type,
   u.name AS veterinarianName,
   b.name AS branchName
FROM Appointment a
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE a.appointmentID = 401
 AND a.petOwnerID = 101;
```

This query supports the header line of the page, such as the visit date, veterinarian name, and clinic branch. It also ensures that the appointment belongs to the currently logged-in pet owner.

Load prescribed medicines

```
SELECT
  p.prescriptionID,
   p.treatment,
   p.prescriptionDate,
   m.medicineID,
   m.name AS medicineName
FROM Prescription p
JOIN Prescribes pr ON pr.prescriptionID = p.prescriptionID
JOIN Medicine m ON m.medicineID = pr.medicineID
WHERE p.veterinarianID = 103
 AND p.petID = 205
ORDER BY p.prescriptionDate DESC;
```

This query is used for the “Prescriptions” section. It combines prescription information with medicine names by joining Prescription, Prescribes, and Medicine. Since the current schema does not have separate dosage and duration attributes, the page mainly shows the prescribed medicine and the treatment description.

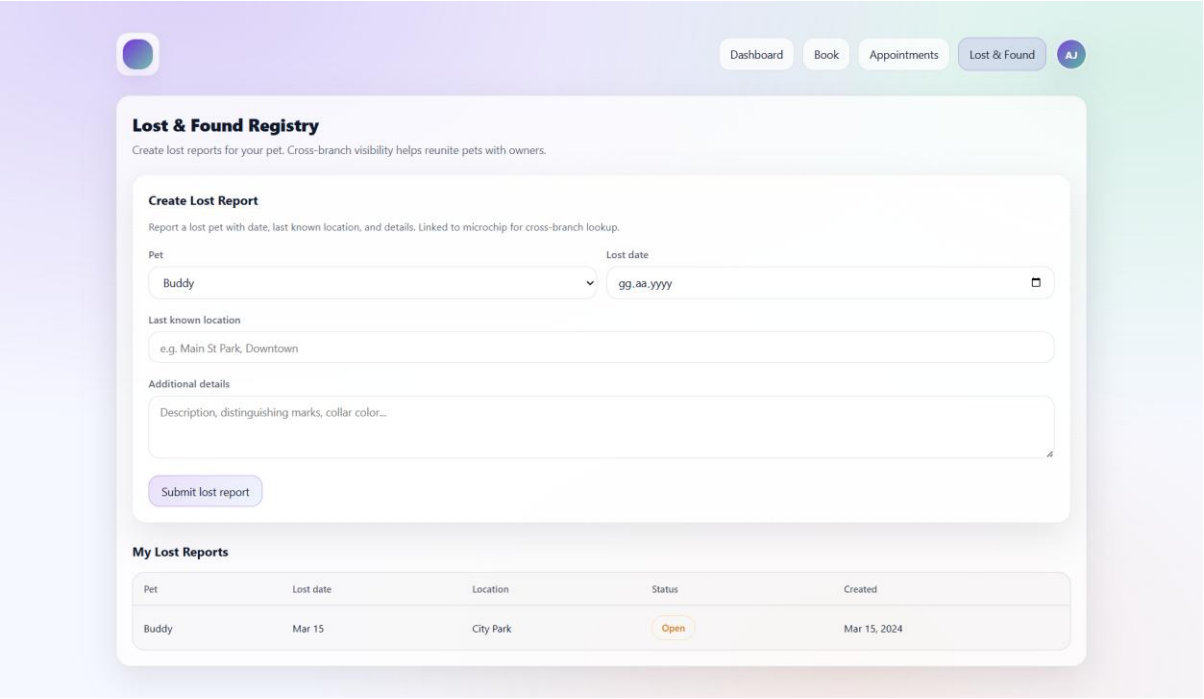
Submit veterinarian rating

After reviewing the completed visit, the pet owner can rate the veterinarian.

```
INSERT INTO Rates (ownerID, veterinarianID, date, rating)
VALUES (101, 103, '2026-03-10', 5);
```

This query stores the owner's rating for the veterinarian in the Rates table. In the current schema, ratings are recorded numerically and linked to the pet owner, veterinarian, and date.

3.2.5 View Lost & Found Reports



This page supports the additional lost-and-found functionality of the system. It allows pet owners to create a lost report for one of their pets and to view previously submitted lost/found records. In the current schema, this functionality is mainly supported by the Pet, Chip, and LostFoundReport tables. The owner selects one of their pets, enters the lost date, and submits a report. The system stores the report together with the report creation date and updates the corresponding chip record so that the pet becomes visible as lost across branches.

Load the pet owner's pets

When the page is opened, the system first retrieves the pets belonging to the currently logged-in owner.

```
SELECT petID, name, species, breed
FROM Pet
WHERE ownerID = 101
ORDER BY name;
```

This query populates the pet selection dropdown in the "Create Lost Report" section.

Load chip information of the selected pet

Before creating the report, the system may retrieve the selected pet's chip information.

```
SELECT chipID, petID, location, isLost, implantationDate, veterinarianID
FROM Chip
WHERE petID = 205;
```

This query retrieves the chip record of the selected pet. The location field can be used for the "last known location" input shown in the interface.

Create a lost report

When the owner submits the form, the system inserts a new lost report with the lost date and report creation date.

```
INSERT INTO LostFoundReport (reportID, isFound, petID, lostDate, reportCreatedDate)
VALUES (301, FALSE, 205, '2026-03-15', '2026-03-15');
```

This query creates a new report in the LostFoundReport table. Since the pet is currently missing, isFound is set to FALSE.

Mark the pet's chip as lost

```
UPDATE Chip
SET isLost = TRUE,
   location = 'City Park, Ankara'
WHERE petID = 205;
```

This query marks the chip as lost and updates the latest known location of the pet.

View the owner's lost/found reports

The page also lists all reports submitted for the current owner's pets.

```
SELECT
  lfr.reportID,
   p.name AS petName,
   lfr.lostDate,
   c.location,
   lfr.reportCreatedDate,
   lfr.isFound
FROM LostFoundReport lfr
JOIN Pet p ON p.petID = lfr.petID
LEFT JOIN Chip c ON c.petID = p.petID
WHERE p.ownerID = 101
ORDER BY lfr.reportCreatedDate DESC;
```

This query fills the “My Lost Reports” table. It shows the pet name, lost date, location, creation time of the report, and whether the pet has been found.

Mark a report as found

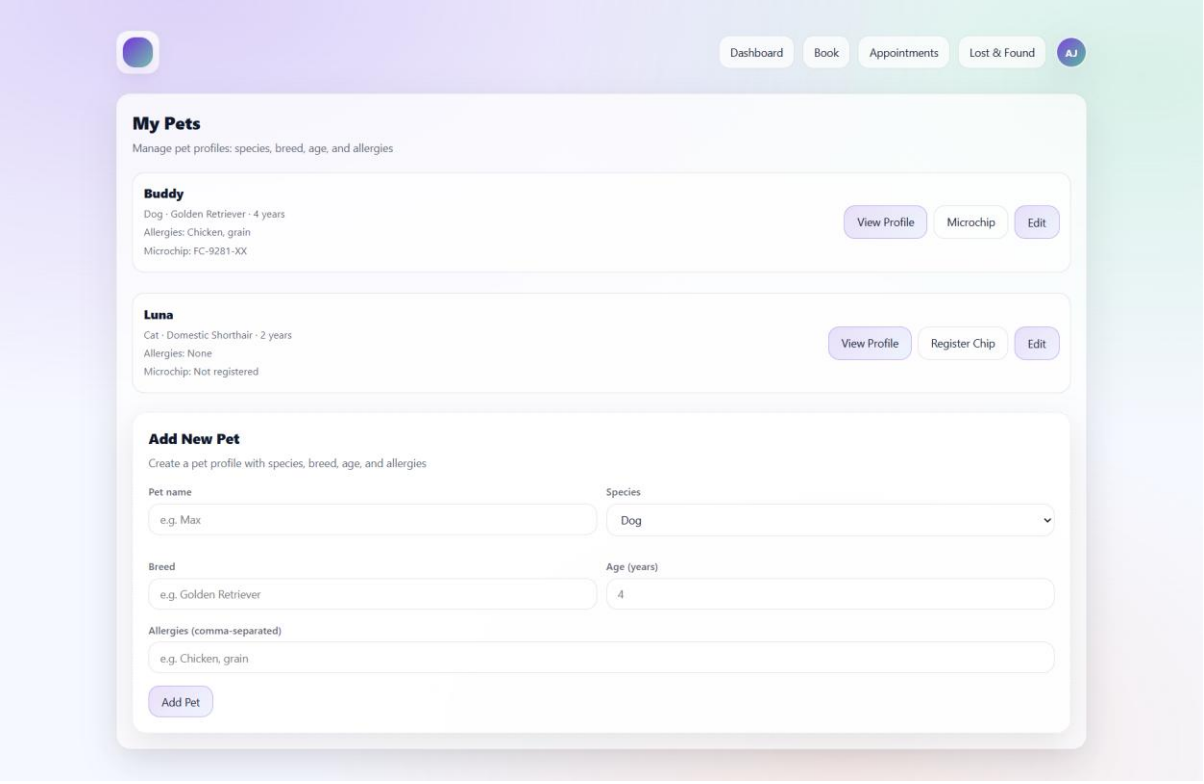
If the pet is found later, the report and chip status can both be updated.

```
UPDATE LostFoundReport
SET isFound = TRUE
WHERE reportID = 301
  AND petID = 205;

UPDATE Chip
SET isLost = FALSE
WHERE petID = 205;
```

These queries close the lost report and reset the chip’s lost status.

3.2.6 Pet Owner Profile Page



This page allows pet owners to manage the profiles of their pets. The owner can view all pets currently registered in the system, inspect profile details such as species, breed, age, allergy-related medical history, and chip information, and add or update pet records. In the current schema, this page is mainly supported by the Pet, MedicalHistory, and Chip tables. Therefore, the page uses SELECT queries to retrieve pet-related data and INSERT / UPDATE queries to create or modify pet profiles. This

page directly supports the pet profile management functionality defined in the project requirements.

Load all pets of the current owner

When the page is opened, the system first retrieves all pets belonging to the logged-in owner.

```
SELECT petID, name, species, breed, age, isAlive
FROM Pet
WHERE ownerID = 101
ORDER BY name;
```

This query fills the “My Pets” section by listing the pets registered under the current owner account. It uses the ownerID foreign key stored in the Pet table.

Load pet profile details together with chip information

```
SELECT
  p.petID,
   p.name,
   p.species,
   p.breed,
   p.age,
   p.sex,
   p.canBreed,
   p.isAlive,
   c.chipID,
   c.implantationDate,
   c.isLost
FROM Pet p
LEFT JOIN Chip c ON c.petID = p.petID
WHERE p.ownerID = 101
ORDER BY p.name;
```

This query is used to show the pet’s main profile data together with microchip availability. If no matching chip row exists, the interface can show a message such as “Microchip: Not registered.”

Add a new pet profile

When the owner fills the “Add New Pet” form, the system inserts a new pet into the Pet table.

```
INSERT INTO Pet
(petID, name, sex, canBreed, age, isAlive, species, breed, ownerID)
```

```
VALUES
(205, 'Buddy', 'M', TRUE, 4, TRUE, 'Dog', 'Golden Retriever', 101);
```

This query creates a new pet profile for the current owner. The pet is linked to the pet owner through ownerID.

Update pet profile information

If the owner edits the profile of an existing pet, the following update query can be used.

```
UPDATE Pet
SET name = 'Buddy',
   species = 'Dog',
   breed = 'Golden Retriever',
   age = 5,
   isAlive = TRUE
WHERE petID = 205
  AND ownerID = 101;
```

This query updates the core pet profile fields while ensuring that the pet belongs to the currently logged-in owner.

Update allergy-related information

If the owner edits known allergies, the system may update the corresponding medical history record.

```
UPDATE MedicalHistory
SET allergies = 'Chicken, grain, beef'
WHERE petID = 205
  AND pastDiagnosis = 'Initial profile entry'
  AND allergies = 'Chicken, grain';
```

This query updates an existing allergy-related medical history entry for the selected pet. Since MedicalHistory uses a composite primary key, the update condition should match the existing record carefully.

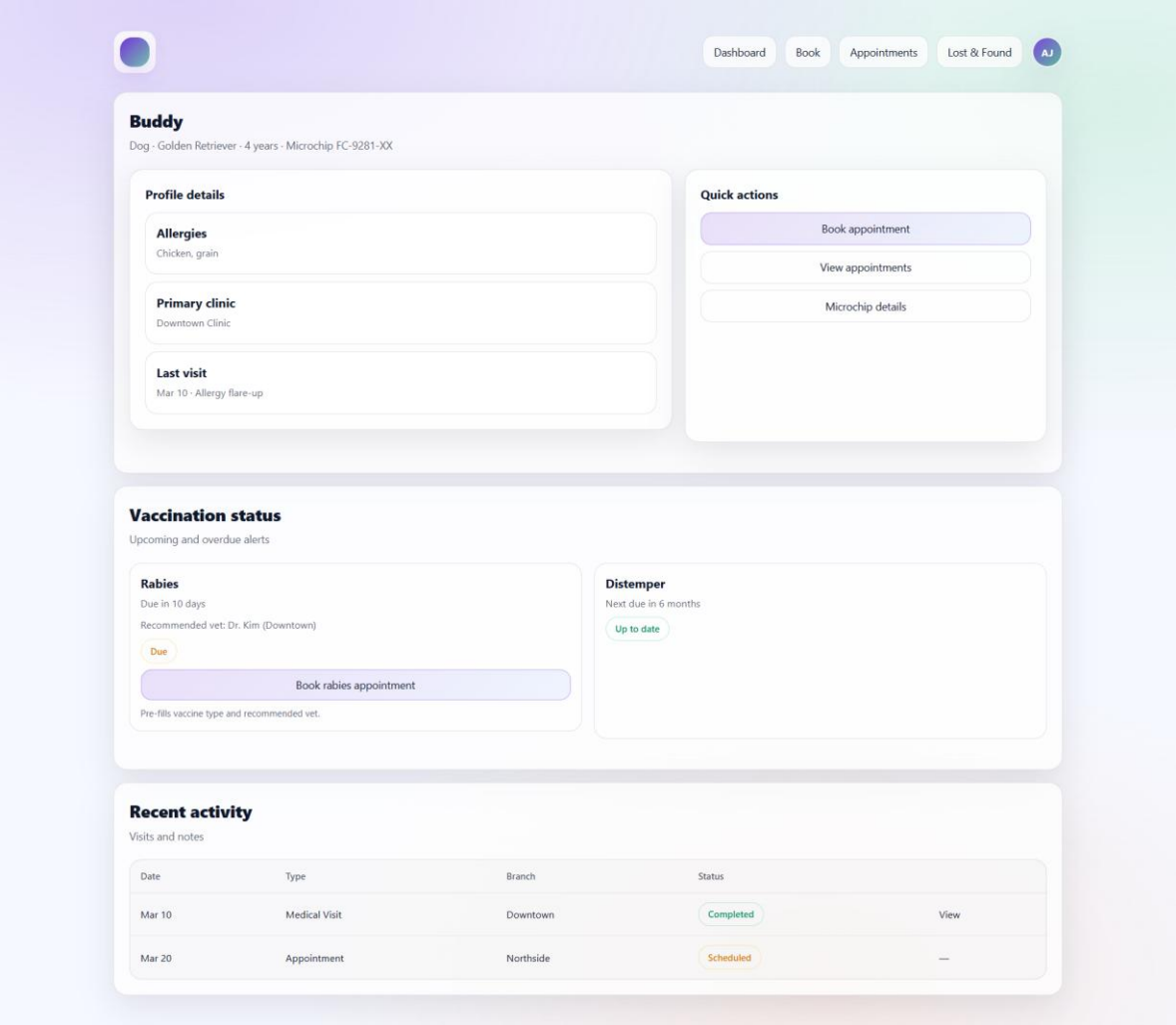
Register a chip for a pet

If the pet does not already have a chip, the following insert operation can be used.

```
INSERT INTO Chip
(chipID, location, isLost, petID, veterinarianID, implantationDate)
VALUES
(92810001, 'Bilkent, Ankara', FALSE, 206, 103, '2026-03-22');
```

This query creates a new chip record for the selected pet. It links the chip to both the pet and the veterinarian who implanted or registered it.

3.2.6.1 View Pet Profile



This page allows the pet owner to view the detailed profile of a selected pet. The page focuses on the pet's identity information, allergy-related medical history, vaccination status, recent activity, and chip information. In the current schema, this functionality is mainly supported by the Pet, MedicalHistory, VaccinationPlan, VaccinationRecord, Chip, Appointment, and VisitSummary tables. Therefore, this page mainly uses SELECT queries to retrieve the selected pet's information.

Load selected pet profile details

When the owner opens a specific pet profile, the system retrieves the pet's basic information together with chip information.

```
SELECT
  p.petID,
   p.name,
   p.species,
```

```
  p.breed,
  p.age,
  c.chipID,
  c.location,
  c.isLost
FROM Pet p
LEFT JOIN Chip c ON c.petID = p.petID
WHERE p.petID = 205
 AND p.ownerID = 101;
```

This query is used for the header and profile details section of the page. It displays the selected pet's identity information and whether a chip is registered.

Load allergy information of the pet

Since allergy data is stored in the medical history table, the system retrieves allergy-related records separately.

```
SELECT pastDiagnosis, allergies
FROM MedicalHistory
WHERE petID = 205;
```

Load vaccination status

The vaccination area can be populated using the pet's vaccination plan and next due dates.

```
SELECT
  vp.planID,
   vp.nextVaccinationDate
FROM VaccinationPlan vp
WHERE vp.petID = 205
ORDER BY vp.nextVaccinationDate ASC;
```

This query supports the vaccination status cards such as upcoming or overdue vaccines.

Load recent vaccination records

```
SELECT
  vr.recordID,
   vr.shotDate,
   vr.nextDueDate
FROM VaccinationRecord vr
WHERE vr.petID = 205
ORDER BY vr.shotDate DESC;
```

This query shows recent vaccination history of the selected pet.

Load recent activity

The recent activity section can display appointments and documented visit summaries related to the pet's owner.

```
SELECT
  a.appointmentID,
   a.dateTime,
   a.type
FROM Appointment a
WHERE a.petOwnerID = 101
ORDER BY a.dateTime DESC;
```

This query provides recent appointment activity visible on the pet profile page.

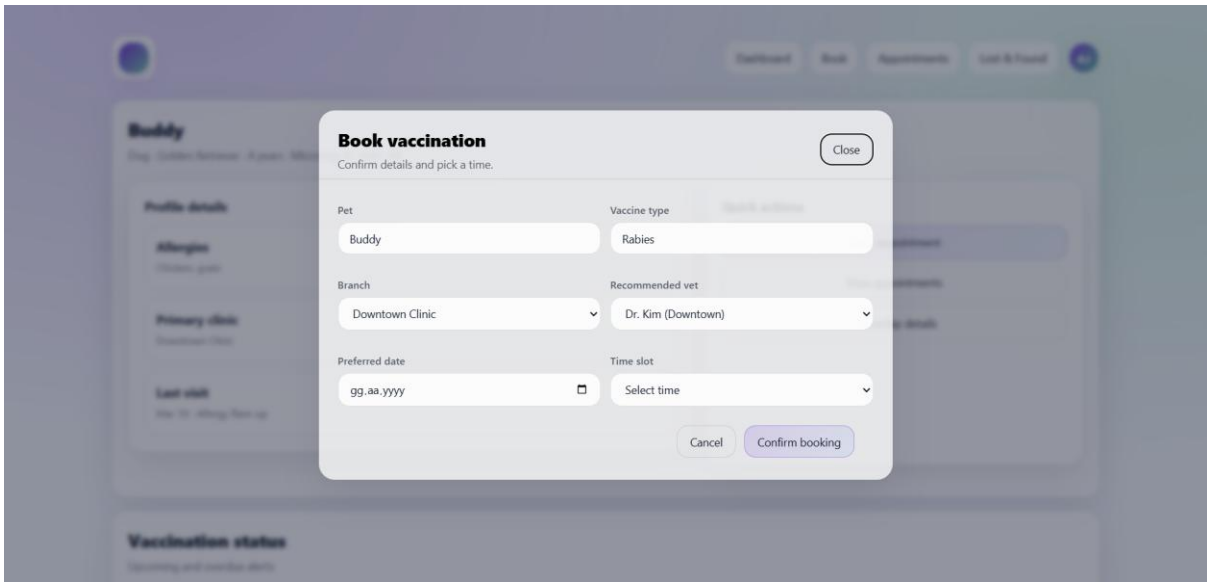
Load microchip details

If the owner clicks the "Microchip details" action, the system can retrieve the full chip record.

```
SELECT
  chipID, location, isLost, implantationDate, veterinarianID
FROM Chip
WHERE petID = 205;
```

This query is used for the microchip details section or popup.

3.2.6.2 Book Vaccination Appointment With Recommended Veterinarian



This page is a follow-up action of the **View Pet Profile** page. After the pet owner

views the selected pet's profile and vaccination status in Section **3.2.6.1**, the system may suggest booking a vaccination appointment for an upcoming or overdue vaccine. Therefore, this interface is not an isolated module; it is directly connected to the vaccination status panel of the selected pet profile. The owner can choose a branch, view the recommended veterinarian for the vaccine, select a preferred date and time slot, and confirm the appointment. In the current schema, this functionality is mainly supported by the VaccinationPlan, Recommended, Veterinarian, User, Branch, and Appointment tables.

Load the vaccination plan of the selected pet

Since this popup is opened from the selected pet profile, the system first retrieves the vaccination plan of that pet.

```
SELECT
  vp.planID,
   vp.nextVaccinationDate,
   vp.petID,
   vp.veterinarianID
FROM VaccinationPlan vp
WHERE vp.petID = 205
ORDER BY vp.nextVaccinationDate ASC;
```

This query connects the popup to the previous page by using the pet selected in the **View Pet Profile** interface. It allows the application to determine which vaccination plan is currently due or approaching.

Load recommended veterinarians for the selected vaccination plan

After identifying the relevant vaccination plan, the system retrieves the recommended veterinarian or veterinarians associated with that plan.

```
SELECT DISTINCT
   r.appointmentID,
   v.veterinarianID,
   u.name AS veterinarianName,
   b.name AS branchName,
   v.speciesExpertise,
   v.availableDates
FROM Appointment a
JOIN Recommended r ON r.appointmentID = a.appointmentID
JOIN Veterinarian v ON v.veterinarianID = r.veterinarianID
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
```

```
WHERE a.vaccinationPlanID = 801
ORDER BY u.name;
```

This query is used to populate the “Recommended vet” field in the booking popup. It directly reflects the recommendation logic initiated from the pet’s vaccination profile.

Check occupied appointment times of the selected veterinarian

Before creating the vaccination booking, the system checks the selected veterinarian’s occupied slots for the chosen date.

```
SELECT appointmentID, dateTime, type
FROM Appointment
WHERE veterinarianID = 103
  AND DATE(dateTime) = '2026-04-05'
ORDER BY dateTime;
```

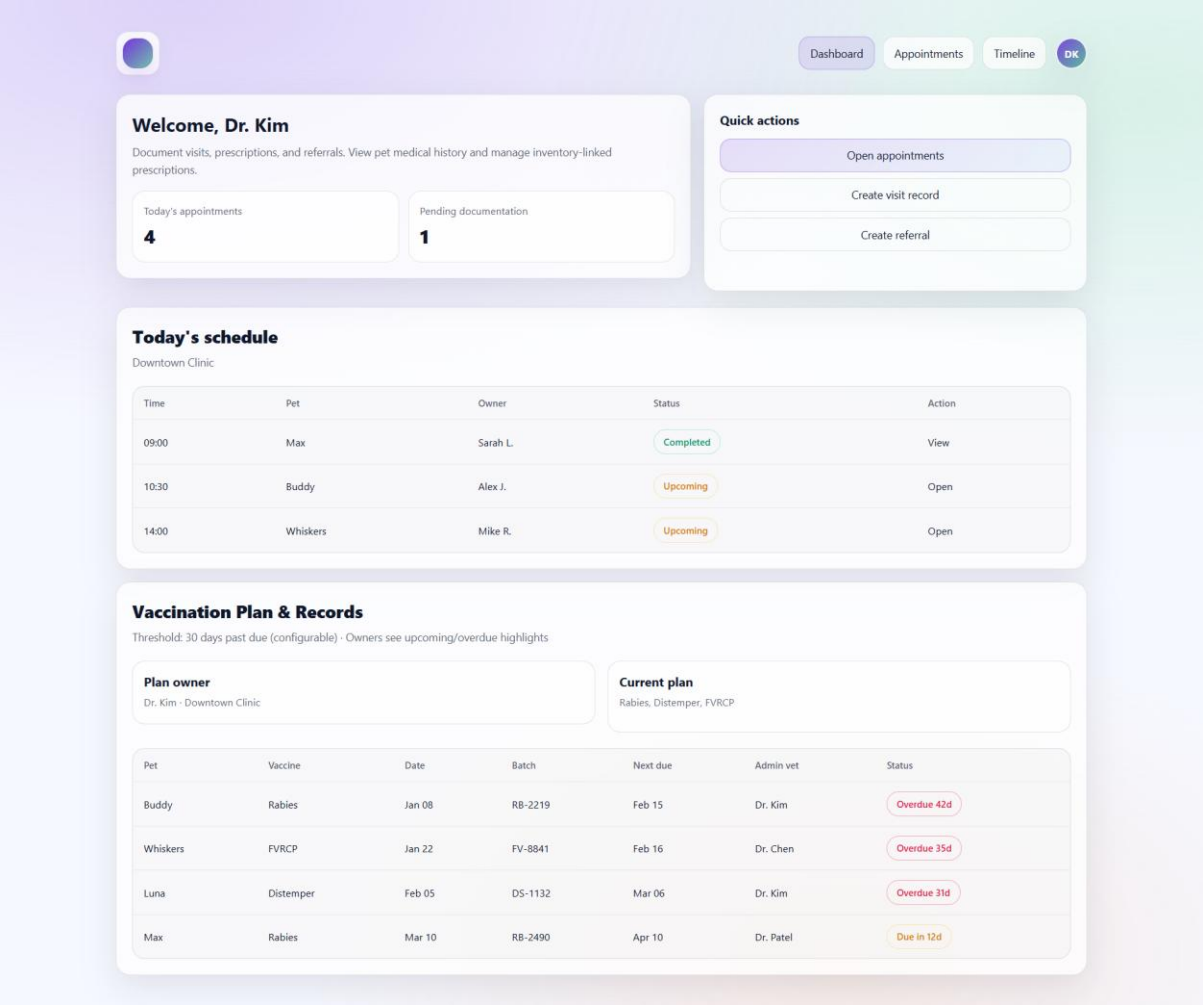
Create the vaccination appointment

After the owner confirms the booking, the system inserts a new appointment linked to the selected vaccination plan.

```
INSERT INTO Appointment
(appointmentID, type, dateTime, vaccinationPlanID, veterinarianID, petOwnerID)
VALUES
(450, 'VACCINATION', '2026-04-05 10:30:00', 801, 103, 101);
```

3.3 Pages Accessible by Veterinarian

3.3.1.1 Veterinarian Dashboard



This page is the main overview screen for veterinarians after login. It summarizes the veterinarian's profile, today's appointment load, pending documentation count, and daily schedule. The dashboard mainly uses SELECT queries to retrieve appointment and profile information for the currently logged-in veterinarian. It may also provide quick access to actions such as opening appointments, creating visit records, and preparing referrals. In the current schema, this page is mainly supported by the User, Veterinarian, Branch, Appointment, and VisitSummary tables.

Load veterinarian profile information

This query retrieves the logged-in veterinarian's basic profile and assigned branch.

```
SELECT
  v.veterinarianID,
   u.name AS veterinarianName,
   u.email,
```

```
  u.phoneNumber,
  v.speciesExpertise,
  v.rating,
  v.maxDailyAppointmentLimit,
  b.name AS branchName,
  b.location
FROM Veterinarian v
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE v.veterinarianID = 103;
```

This query is used to display the welcome area of the dashboard, such as “Welcome, Dr. Kim,” together with the veterinarian’s branch and expertise information.

Count today’s appointments

```
SELECT COUNT(*) AS todaysAppointments
FROM Appointment
WHERE veterinarianID = 103
  AND DATE(dateTime) = '2026-03-24';
```

This result is used for the small summary card showing the number of appointments scheduled for today. Since Appointment stores both dateTime and veterinarianID, it is the natural source for daily workload information.

Load today’s schedule

```
SELECT
  a.appointmentID,
  a.dateTime,
  a.type,
  po.ownerID,
  uo.name AS ownerName,
  b.name AS branchName
FROM Appointment a
JOIN PetOwner po ON po.ownerID = a.petOwnerID
JOIN User uo ON uo.userID = po.ownerID
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE a.veterinarianID = 103
  AND DATE(a.dateTime) = '2026-03-24'
ORDER BY a.dateTime ASC;
```

This query fills the “Today’s schedule” table. In the current schema, the appointment is directly linked to the pet owner and the veterinarian, so the schedule can show time, owner, appointment type, and branch information.

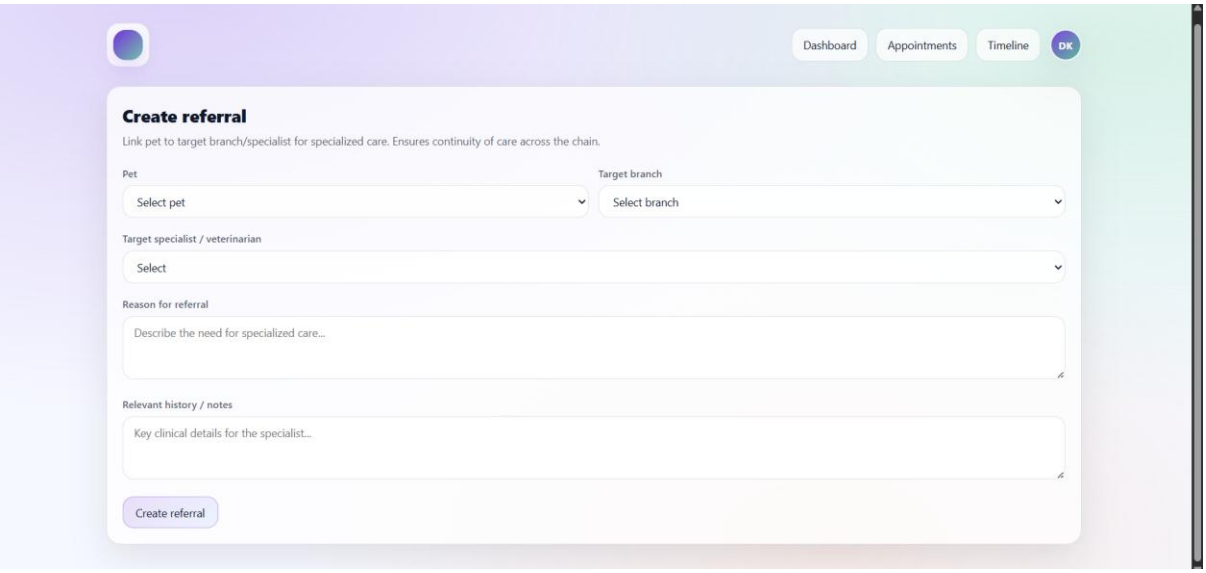
Quick action: prepare referral target list

If the “Create referral” quick action is selected, the veterinarian can view other available veterinarians as possible referees.

```
SELECT
  v.veterinarianID,
   u.name AS veterinarianName,
   b.name AS branchName,
   v.speciesExpertise
FROM Veterinarian v
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE v.veterinarianID <> 103
ORDER BY b.name, u.name;
```

This query prepares the candidate veterinarian list for a referral workflow. Since the current schema models referral through the Refers relation between veterinarians, the system must first retrieve possible target veterinarians.

3.3.1.2 Create Referral



This page allows a veterinarian to refer a case to another veterinarian when specialized care is needed. In the current schema, referral data is stored through the Refers relation between veterinarians. Therefore, the interface first loads possible target veterinarians together with their branch information, and then creates a referral record

by storing the referrer veterinarian, referee veterinarian, referral date, and diagnosis. This functionality supports continuity of care across the clinic chain.

Load target veterinarians with branch information

```
SELECT
  v.veterinarianID,
   u.name AS veterinarianName,
   b.branchID,
   b.name AS branchName,
   v.speciesExpertise
FROM Veterinarian v
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE v.veterinarianID <> 103
ORDER BY b.name, u.name;
```

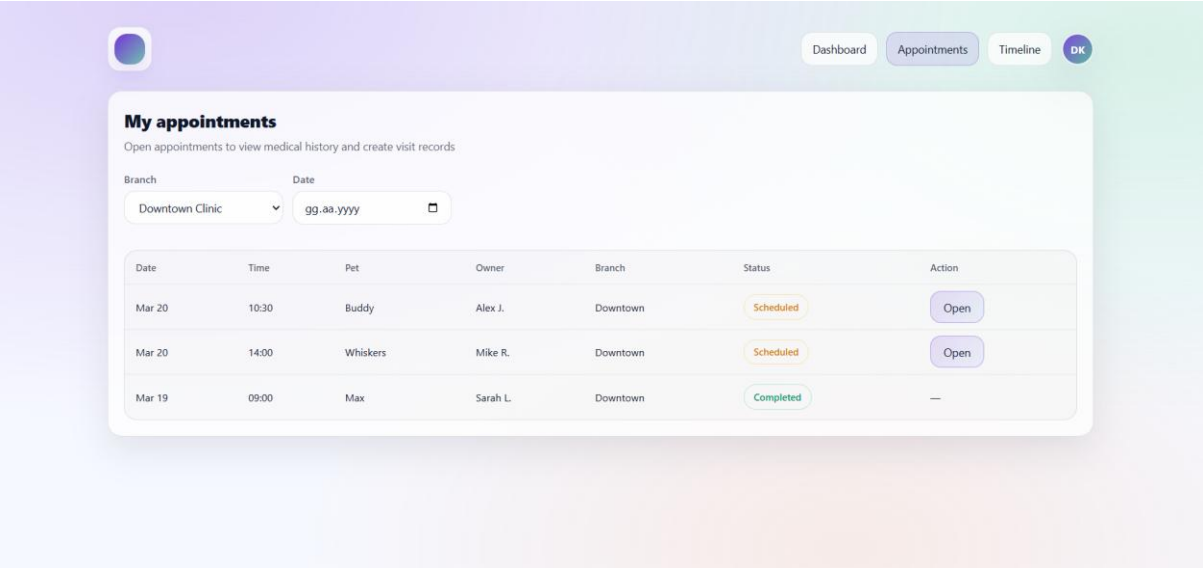
This query fills the target specialist / veterinarian list together with branch information.

Create referral record

```
INSERT INTO Refers (referrer, referee, referralDate, diagnosis)
VALUES (103, 104, '2026-03-24', 'Suspected cardiac issue requiring specialist evaluation');
```

This query creates the referral from the current veterinarian to the selected specialist.

3.3.2 Appointments Page



This page allows a veterinarian to view and filter their appointments by branch and date. It is mainly used to access scheduled visits, inspect the owner information,

and open an appointment before creating a visit record. In the current schema, appointment data is stored in the Appointment table, veterinarian and branch information are stored in Veterinarian and Branch, and owner information is obtained through PetOwner and User. Therefore, this page mainly relies on SELECT queries to list and filter appointments.

Load all appointments of the veterinarian

When the page is opened, the system retrieves all appointments assigned to the logged-in veterinarian.

```
SELECT
  a.appointmentID,
   a.type,
   a.dateTime,
   uo.name AS ownerName,
   b.name AS branchName
FROM Appointment a
JOIN PetOwner po ON po.ownerID = a.petOwnerID
JOIN User uo ON uo.userID = po.ownerID
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE a.veterinarianID = 103
ORDER BY a.dateTime ASC;
```

This query fills the main appointment table of the veterinarian. It shows the appointment date and time together with the related pet owner and branch information.

Filter appointments by branch

```
SELECT
  a.appointmentID,
   a.type,
   a.dateTime,
   uo.name AS ownerName,
   b.name AS branchName
FROM Appointment a
JOIN PetOwner po ON po.ownerID = a.petOwnerID
JOIN User uo ON uo.userID = po.ownerID
JOIN Veterinarian v ON v.veterinarianID = a.veterinarianID
JOIN Branch b ON b.branchID = v.branchID
WHERE a.veterinarianID = 103
  AND b.branchID = 1
ORDER BY a.dateTime ASC;
```

This query supports the branch filter shown at the top of the page. Since each veterinarian is linked to a branch, branch-based filtering can be performed through the Veterinarian table.

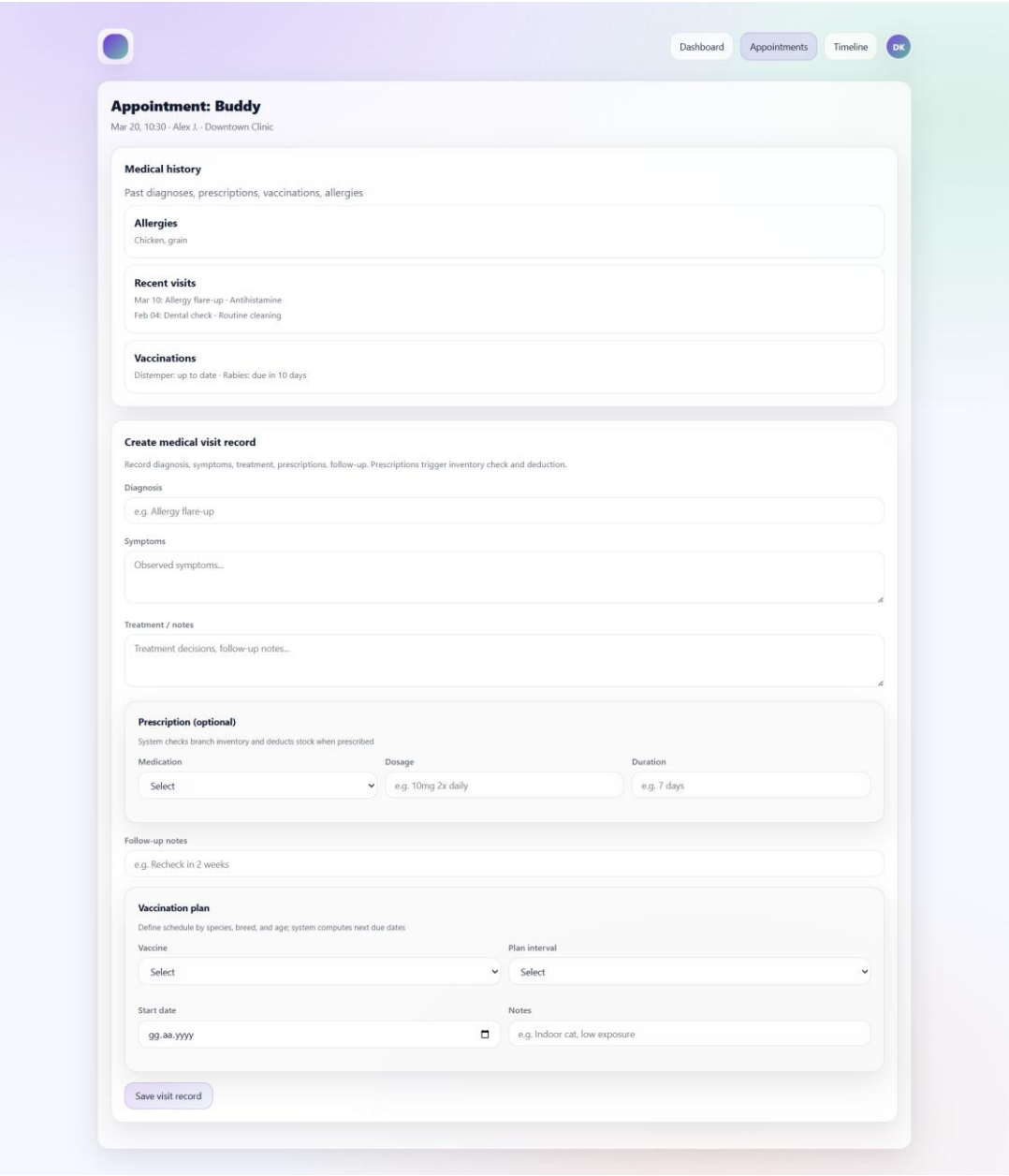
Open a selected appointment

When the veterinarian clicks the “Open” button, the system retrieves the selected appointment record.

```
SELECT
  a.appointmentID,
   a.type,
   a.dateTime,
   a.vaccinationPlanID,
   a.veterinarianID,
   a.petOwnerID
FROM Appointment a
WHERE a.appointmentID = 401
  AND a.veterinarianID = 103;
```

This query is used before navigating to the detailed appointment page. It ensures that the selected appointment belongs to the currently logged-in veterinarian.

3.3.3 Detailed Appointment View



This page is used by the veterinarian after opening an appointment. It allows the veterinarian to review the pet's medical history, allergies, recent visits, and vaccination records, and then create a new medical visit record. The page also supports optional prescription creation, inventory-linked medicine deduction, and vaccination plan definition. These functionalities are supported mainly by MedicalHistory, Prescription, Prescribes, Medicine, VaccinationPlan, VaccinationRecord, VisitSummary, and Appointment.

Load pet medical history

```
SELECT petID, pastDiagnosis, allergies
FROM MedicalHistory
WHERE petID = 205;
```

This query is used to populate the “Medical history” section, especially the allergy and diagnosis-related information. The MedicalHistory table stores the historical clinical background of the pet.

Load medicines used in a previous prescription

This query shows which medicines were prescribed in a selected prescription.

```
SELECT
  p.prescriptionID,
   m.medicineID,
   m.name,
   m.status,
   m.quantity
FROM Prescribes pr
JOIN Prescription p ON p.prescriptionID = pr.prescriptionID
JOIN Medicine m ON m.medicineID = pr.medicineID
WHERE p.prescriptionID = 601;
```

This query is useful when the veterinarian wants to inspect earlier medication choices for the pet. It combines Prescribes, Prescription, and Medicine.

Load vaccination history

```
SELECT
  vr.recordID,
   vr.shotDate,
   vr.frequency,
   vr.nextDueDate,
   vr.pastDiagnosis,
   vr.allergies
FROM VaccinationRecord vr
WHERE vr.petID = 205
ORDER BY vr.shotDate DESC;
```

This query supports the vaccination section of the page by listing previous vaccine records and the next due dates.

Save visit summary

After examining the pet, the veterinarian can save diagnosis and treatment notes for the appointment.

```
INSERT INTO VisitSummary (appointmentID, notes)
VALUES (
  401,
  'Allergy flare-up. Symptoms: itching and sneezing. Prescribed antihistamine treatment and follow-up in 2 weeks.'
);
```

This query creates the visit summary of the appointment. In the current schema, the textual summary is stored in the notes field of VisitSummary.

Create a new prescription

```
INSERT INTO Prescription
(prescriptionID, treatment, veterinarianID, petID, pastDiagnosis, allergies, prescriptionDate)
VALUES
(602, 'Antihistamine 10mg twice daily for 7 days', 103, 205, 'Allergy flare-up', 'Chicken, grain', '2026-03-20');
```

This query stores the prescribed treatment for the pet. The prescription is linked both to the veterinarian and to the pet's medical history through (petID, pastDiagnosis, allergies).

Link the prescription to a medicine

```
INSERT INTO Prescribes (prescriptionID, medicineID)
VALUES (602, 301);
```

This query records which medicine belongs to the new prescription.

Deduct medicine stock

```
UPDATE Medicine
SET quantity = quantity - 1
WHERE medicineID = 301
  AND branchID = 1
  AND quantity > 0;
```

This query performs the stock deduction step after prescription creation. In the proposal, prescription creation and stock deduction are part of the same workflow.

Create a vaccination plan

If the veterinarian decides that a vaccination schedule is needed, a new plan can be defined.

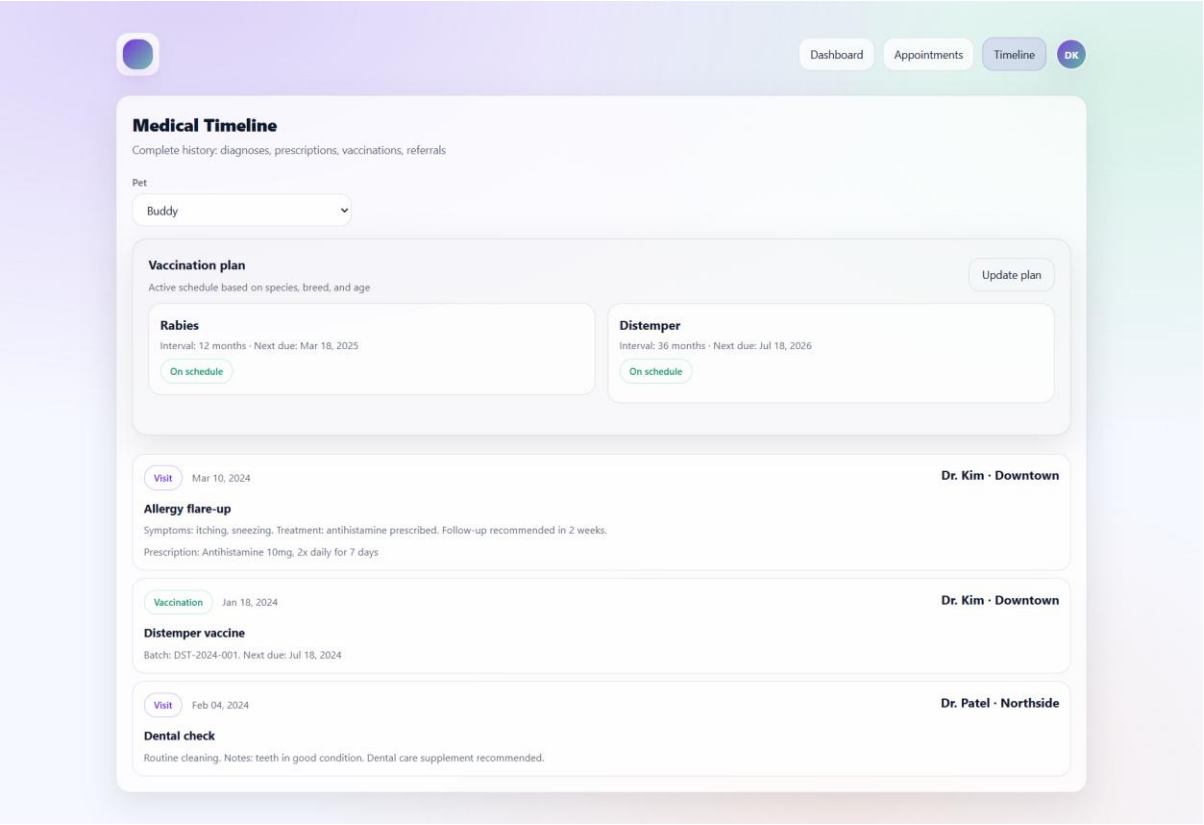
```
INSERT INTO VaccinationPlan (planID, nextVaccinationDate, petID, veterinarianID)
VALUES (801, '2026-09-20', 205, 103);
```

Create a vaccination record

If a vaccine is administered during the visit, a vaccination record can also be created.

```
INSERT INTO VaccinationRecord
(recordID, threshold, shotDate, frequency, nextDueDate, planID, petID, pastDiagnosis, allergies)
VALUES
(701, 30, '2026-03-20', '6 months', '2026-09-20', 801, 205, 'Allergy flare-up', 'Chicken, grain');
```

3.3.4 Medical Timeline



This page allows the veterinarian to view the complete medical history of a selected pet in a timeline format. It combines vaccination planning, previous visit-related prescriptions, vaccination records, and referral-related information in one interface. Since the current schema stores these data in separate tables, the timeline is generated by retrieving records from multiple sources and ordering them chronologically at the application level.

Load selected pet list

Before displaying the timeline, the system retrieves pets that can be selected by the veterinarian.

```
SELECT petID, name, species, breed, age
FROM Pet
ORDER BY name;
```

This query populates the pet dropdown at the top of the timeline page.

Load vaccination plan of the selected pet

```
SELECT
  vp.planID,
   vp.nextVaccinationDate,
   vp.veterinarianID
FROM VaccinationPlan vp
WHERE vp.petID = 205
ORDER BY vp.nextVaccinationDate ASC;
```

This query is used for the vaccination plan cards shown at the top of the page.

Load prescription-based visit history

Previous medical visits can be represented through prescription records linked to the selected pet.

```
SELECT
  p.prescriptionID,
   p.prescriptionDate,
   p.treatment,
   p.pastDiagnosis,
   u.name AS veterinarianName,
   b.name AS branchName
FROM Prescription p
JOIN Veterinarian v ON v.veterinarianID = p.veterinarianID
JOIN User u ON u.userID = v.veterinarianID
LEFT JOIN Branch b ON b.branchID = v.branchID
WHERE p.petID = 205
ORDER BY p.prescriptionDate DESC;
```

This query provides timeline entries for diagnosis, treatment, and prescription-related visits.

Load referral-related history

If the page also displays referred cases, the system can retrieve referral records connected to veterinarians involved in the pet's treatment history.

```
SELECT
  r.referralDate,
   r.diagnosis,
   ur.name AS referrerName,
   ue.name AS refereeName
FROM Refers r
JOIN User ur ON ur.userID = r.referrer
JOIN User ue ON ue.userID = r.referee
ORDER BY r.referralDate DESC;
```

This query can be used to display referral events as part of the broader clinical timeline.

Update vaccination plan

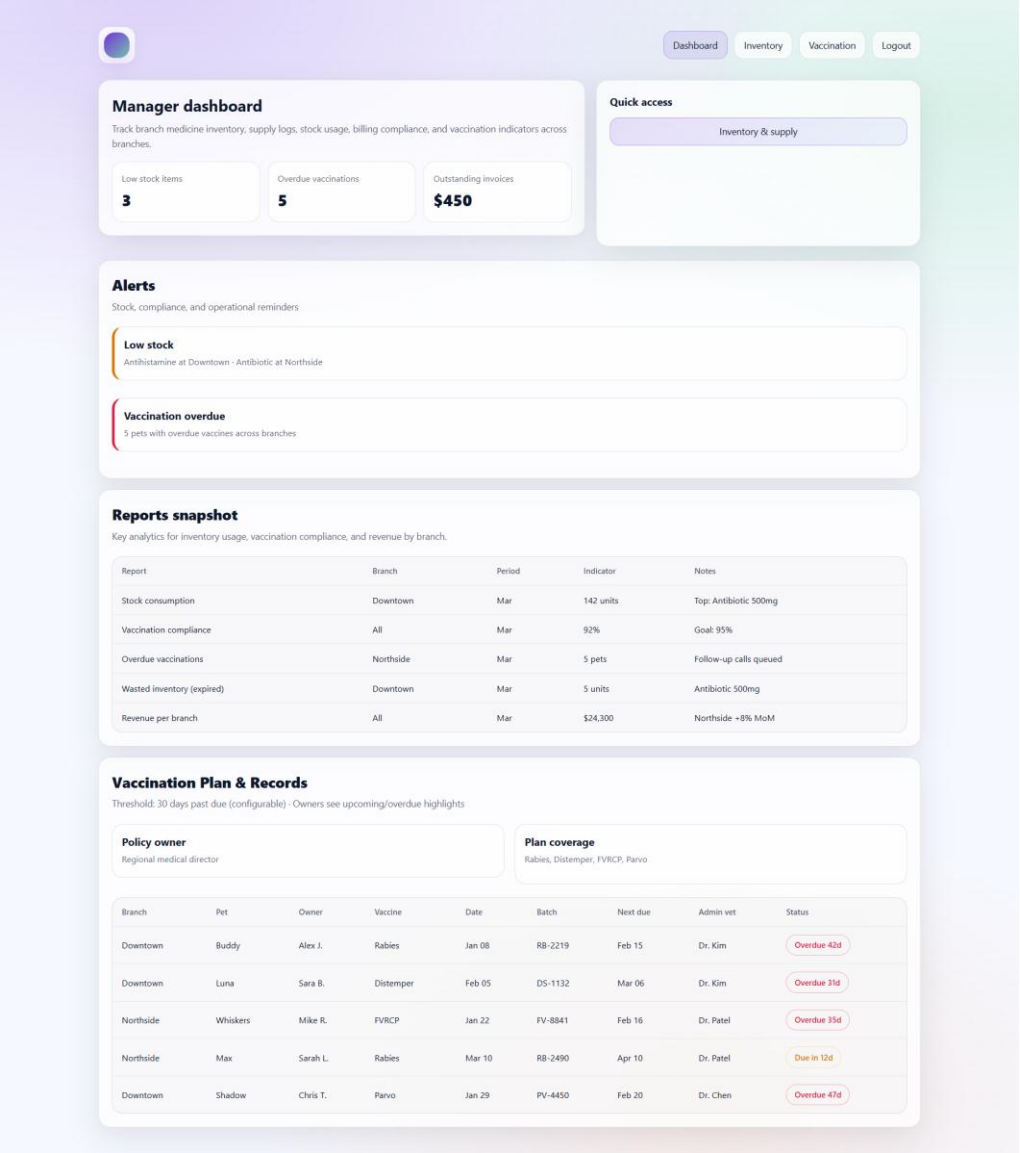
If the veterinarian clicks the “Update plan” action, the vaccination schedule can be updated.

```
UPDATE VaccinationPlan
SET nextVaccinationDate = '2026-11-18'
WHERE planID = 801
  AND petID = 205;
```

This query modifies the selected vaccination plan and represents the editable part of the timeline page.

3.4 Pages Accessible by Clinic Manager

3.4.1 Clinic Manager Dashboard



This page provides a summary view for the clinic manager. It shows branch-level inventory alerts, overdue vaccination indicators, unpaid billing totals, and vaccination record highlights. In the current schema, this page mainly uses Medicine, Bill, VaccinationPlan, VaccinationRecord, Pet, User, and Branch tables. Therefore, the dashboard mainly relies on SELECT queries for summary reporting.

Low stock items

The manager dashboard shows how many medicines in a branch are currently at or below their threshold level.

```
SELECT COUNT(*) AS lowStockCount
FROM Medicine
WHERE branchID = 1
```

```
  AND quantity <= threshold
  AND status = 'safe';
```

This query is used for the “Low stock items” summary card. It helps the manager quickly identify whether the branch needs restocking action.

Reports snapshot

The reports snapshot section presents a compact operational summary for the manager, such as inventory usage, vaccination compliance, overdue cases, and branch-level activity.

```
SELECT
  m.branchID,
   COUNT(*) AS lowStockMedicines,
   SUM(m.quantity) AS totalUnitsInStock
FROM Medicine m
GROUP BY m.branchID
ORDER BY m.branchID;
```

This query provides a simple inventory-oriented report summary by branch.

```
SELECT
  COUNT(*) AS totalVaccinationRecords,
   SUM(CASE WHEN nextDueDate < '2026-03-24' THEN 1 ELSE 0 END) AS
overdueVaccinationRecords
FROM VaccinationRecord;
```

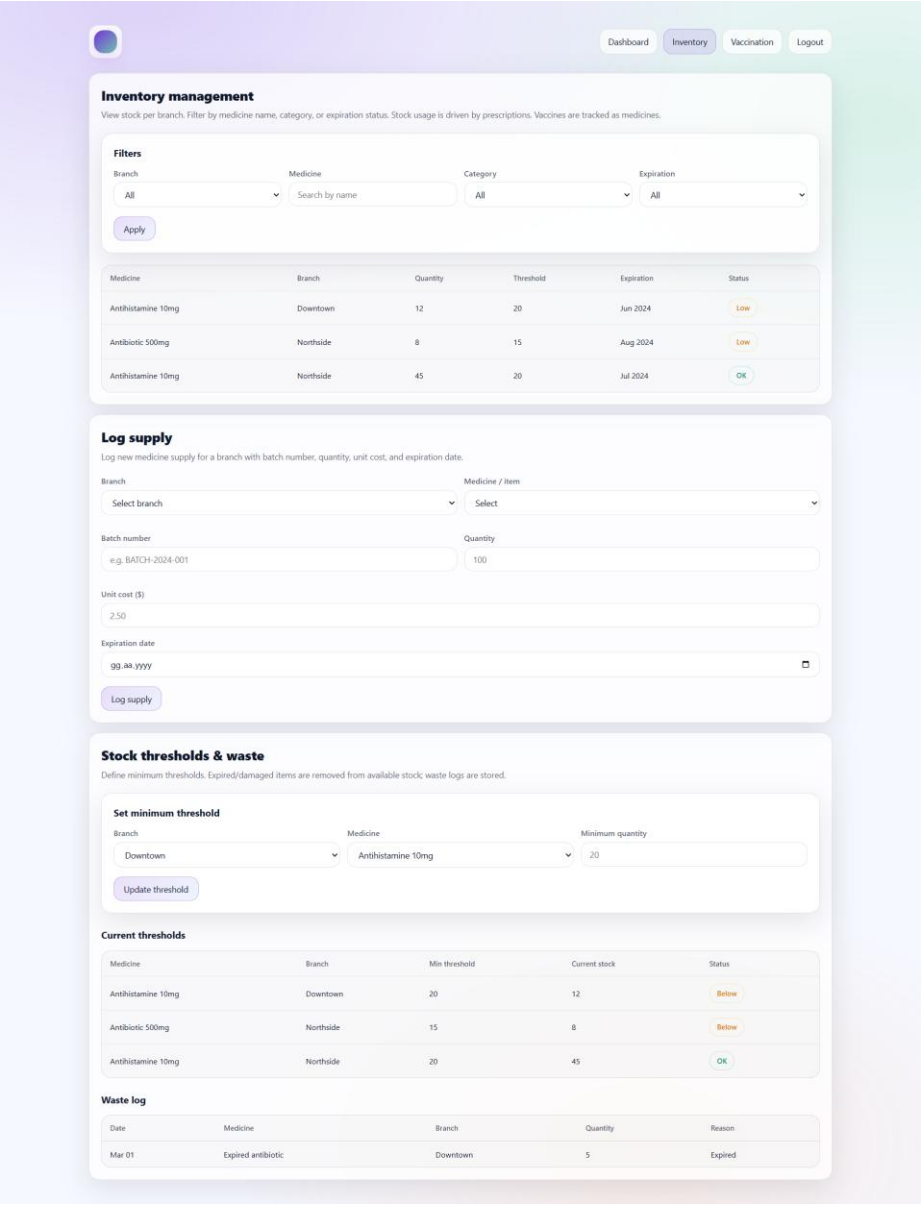
This query gives a compact vaccination compliance snapshot, which can be shown in the report summary section.

Vaccination Plan & Records

```
SELECT
  vr.recordID,
   p.name AS petName,
   u.name AS ownerName,
   vr.shotDate,
   vr.nextDueDate
FROM VaccinationRecord vr
JOIN Pet p ON p.petID = vr.petID
JOIN PetOwner po ON po.ownerID = p.ownerID
JOIN User u ON u.userID = po.ownerID
ORDER BY vr.nextDueDate ASC;
```

This query is used to populate the vaccination tracking table. It allows the manager to see which pets are close to or past their next due vaccination dates.

3.4.2 Inventory



This page allows the clinic manager to monitor branch-based medicine inventory, log new supply entries, define minimum stock thresholds, and review waste records. Since the updated system design stores branch-specific stock directly in the Medicine table, the inventory management page is mainly built on Medicine and WasteLog. The page includes both retrieval and modification operations: SELECT queries are used to display stock information and waste history, while INSERT and UPDATE queries are used to log new supply entries and update stock thresholds.

Filter inventory by medicine name

If the manager searches by medicine name, the system retrieves matching rows.

```
SELECT
  medicineID,
   name,
   branchID,
   quantity,
   threshold,
   expiracyDate,
   status
FROM Medicine
WHERE name LIKE '%Antihistamine%'
ORDER BY branchID, name;
```

This query supports the medicine search box in the inventory filter area.

Log new supply

When the manager enters a new supply record, the medicine quantity can be increased.

```
UPDATE Medicine
SET quantity = quantity + 100,
   expiracyDate = '2026-08-30',
   status = 'safe'
WHERE medicineID = 301
 AND branchID = 1;
```

This query represents the “Log supply” action in the simplest schema-compatible way. Since stock is tracked directly in Medicine, adding supply means updating the quantity of the relevant branch-specific medicine record.

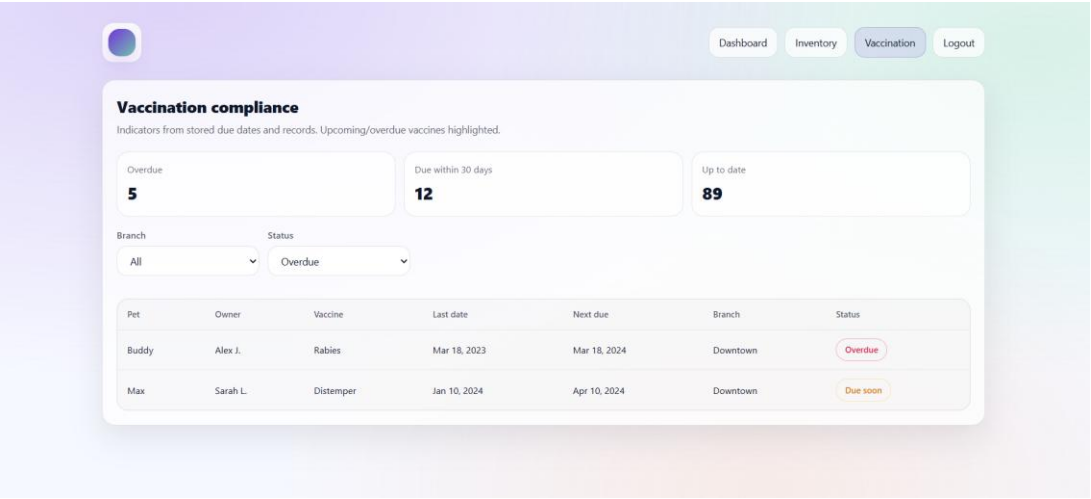
Insert waste log entry

If expired or damaged stock is discarded, the system may record it in the waste log.

```
INSERT INTO WasteLog (logID, medicineID, branchID, quantity, reason, logDate)
VALUES (501, 301, 1, 5, 'Expired', '2026-03-24');
```

This query logs the discarded quantity and reason for removal.

3.4.3 Vaccination



This page allows the clinic manager to monitor vaccination compliance across the system. It highlights overdue vaccinations, vaccinations due soon, and up-to-date records, and it provides a filterable list of pets together with their owners, vaccine dates, and compliance status. In the current schema, this functionality is mainly supported by the VaccinationRecord, Pet, PetOwner, User, and VaccinationPlan tables. Therefore, the page mainly uses SELECT queries to retrieve vaccination monitoring data.

Count vaccinations due within 30 days

The second summary card shows vaccinations that are not overdue yet but will become due soon.

```
SELECT COUNT(*) AS dueWithin30Days
FROM VaccinationRecord
WHERE nextDueDate >= '2026-03-24'
  AND nextDueDate <= '2026-04-23';
```

This query is used for the “Due within 30 days” summary card.

Load compliance status as a readable label

If the interface wants to show a textual status such as “Overdue,” “Due soon,” or “Up to date,” it can be derived from nextDueDate.

```
SELECT
  vr.recordID,
   p.name AS petName,
   u.name AS ownerName,
   vr.shotDate,
   vr.nextDueDate,
   CASE
      WHEN vr.nextDueDate < '2026-03-24' THEN 'Overdue'
```

```
    WHEN vr.nextDueDate <= '2026-04-23' THEN 'Due soon'
    ELSE 'Up to date'
  END AS complianceStatus
FROM VaccinationRecord vr
JOIN Pet p ON p.petID = vr.petID
JOIN PetOwner po ON po.ownerID = p.ownerID
JOIN User u ON u.userID = po.ownerID
ORDER BY vr.nextDueDate ASC;
```

This query supports the status labels shown in the table.

4. Advanced Database Components

This section presents the advanced database components planned for the Veterinary Clinic Chain Management System. In accordance with the design report requirements, the database design includes views, triggers, and constraints in order to enforce business rules at the database level and support role-specific interfaces. These components are especially important in our project because the system contains rule-based workflows such as appointment validation, vaccination monitoring, medicine stock control, and automatic billing generation.

4.1 Views

4.1.1 Upcoming and Overdue Vaccinations

This view is created to support vaccination tracking and compliance monitoring. Pet owners should be able to see whether a vaccination is upcoming or overdue when they view the pet profile, and managers should be able to monitor overdue vaccination statistics across the clinic chain. Since this status is computed from stored dates, a view is an appropriate database component for this purpose. This also directly matches the project requirement that overdue and upcoming vaccinations should be highlighted through a query or view.

This view is used in the pet owner interface and manager dashboard. It allows the application to retrieve a ready-made vaccination status without recalculating due-state logic repeatedly in different pages.

```
CREATE VIEW VaccinationStatusView AS
SELECT
  vr.recordID,
  vr.petID,
```

```
  p.name AS petName,
  vr.shotDate,
  vr.nextDueDate,
  m.name AS vaccineName,
  CASE
    WHEN vr.nextDueDate < CURRENT_DATE THEN 'Overdue'
    WHEN vr.nextDueDate <= CURRENT_DATE + INTERVAL 30 DAY THEN 'Upcoming'
    ELSE 'Normal'
  END AS vaccinationStatus
FROM VaccinationRecord vr
JOIN Pet p ON vr.petID = p.petID
JOIN Involves i ON vr.recordID = i.recordID
JOIN Vaccine v ON i.vaccineID = v.vaccineID
JOIN Medicine m ON v.vaccineID = m.medicineID;
```

4.1.2 Low Stock Medicines by Branch

This view is created to support medicine and supply management. In the revised design, medicine stock is held directly per branch, and the manager is responsible for manually logging new supply. Since the manager must detect low-stock medicines after prescriptions or waste logging, a dedicated view is useful for monitoring medicines whose quantity has fallen below the threshold. This matches the requirement that low-stock alerts should be flagged when stock falls below threshold.

This view is used in the clinic manager interface to show medicines that need urgent attention. It simplifies the manager dashboard and can also be used in reporting screens for stock alerts.

```
CREATE VIEW LowStockMedicineView AS
SELECT
  medicineID,
  name AS medicineName,
  branchID,
  quantity,
  threshold,
  status,
  expiracyDate
FROM Medicine
WHERE quantity < threshold
 AND status = 'safe';
```

4.1.3 Unpaid Bills of Pet Owners

This optional third view supports appointment validation. According to the project requirements, a booking should be blocked if the owner has unpaid bills. Since the booking page repeatedly checks this rule, a reusable view simplifies the logic and makes the validation more readable.

This view is used during the appointment booking workflow. Before a new appointment is created, the system checks whether the current owner appears in this view.

```
CREATE VIEW UnpaidBillsView AS
SELECT
  payerID,
  billNo,
  appointmentID,
  dueDate,
  consultationFee,
  treatmentCost,
  medicationCost
FROM Bill
WHERE paid = FALSE;
```

4.2 Triggers

4.2.1 Prevent Negative Medicine Stock

Prescription creation is followed by stock deduction in current user interface flow, and the project requirements explicitly state that medicine quantity must never become negative. Since Medicine.quantity stores the current stock, the database should prevent invalid updates at the data level.

This trigger is activated whenever stock is updated, especially after prescription deduction or waste-related updates. It guarantees that the database itself rejects invalid negative stock values.

```
CREATE TRIGGER trg_prevent_negative_stock
BEFORE UPDATE ON Medicine
FOR EACH ROW
BEGIN
  IF NEW.quantity < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Medicine stock cannot become negative.';
  END IF;
END;
```

4.2.2 Automatically Deduct Stock When a Prescription Is Linked to a Medicine

In the current schema, the prescription-to-medicine relation is represented by the Prescribes(prescriptionID, medicineID) table, and detailed appointment view page already shows that after creating a prescription, a medicine is linked and then stock is deducted. To support this workflow more safely, an AFTER INSERT trigger on Prescribes can automate the stock reduction.

This trigger is used after the veterinarian creates a prescription and links it to a medicine. It keeps prescription recording and stock updates synchronized at the database level.

```
CREATE TRIGGER trg_deduct_stock_after_prescribes
AFTER INSERT ON Prescribes
FOR EACH ROW
BEGIN
  UPDATE Medicine
  SET quantity = quantity - 1
  WHERE medicineID = NEW.medicineID;
END;
```

4.2.3 Synchronize Lost/Found Report with Chip Status

The additional functionality includes the microchip and lost/found registry, and the current schema contains both LostFoundReport(isFound, petID lostDate, reportCreatedDate) and Chip(isLost, petID, ...). Since these two structures represent related states, a trigger can keep them synchronized.

This trigger supports the additional microchip/lost-found functionality. When a new lost/found report is inserted, the associated chip record is automatically marked as lost or found accordingly.

```
CREATE TRIGGER trg_update_chip_status_after_report
AFTER INSERT ON LostFoundReport
FOR EACH ROW
BEGIN
  UPDATE Chip
  SET isLost = NOT NEW.isFound
  WHERE petID = NEW.petID;
END;
```

4.3 Constraints

4.3.1 No Overlapping Appointments for the Same Veterinarian

One of the most important booking rules of the project is that a veterinarian cannot have overlapping appointments at the same date and time. In the current schema, appointments are stored with veterinarianID and dateTime, so a uniqueness constraint on these attributes is appropriate.

This constraint is enforced during appointment creation. It prevents two bookings from being stored for the same veterinarian at the same date and time.

```
ALTER TABLE Appointment
ADD CONSTRAINT uq_vet_datetime
UNIQUE (veterinarianID, dateTime);
```

4.3.2 A Pet Can Have At Most One Chip

The microchip module requires each pet to have at most one chip record in the registry. In the current schema, Chip references Pet through petID, so a uniqueness constraint on petID is appropriate.

This constraint supports the additional lost/found functionality and ensures that microchip lookup remains unambiguous.

```
ALTER TABLE Chip
ADD CONSTRAINT uq_chip_pet
UNIQUE (petID);
```

4.3.3 Non-Negative Medicine Quantity and Threshold

Medicine stock values and minimum threshold values should never be negative. Since the system uses these attributes for low-stock alerts and deduction logic, a check constraint improves data integrity.

This constraint protects stock data quality and supports safe inventory calculations.

```
ALTER TABLE Medicine
ADD CONSTRAINT chk_medicine_nonnegative
CHECK (quantity >= 0 AND threshold >= 0);
```

4.3.4 Valid Rating Range

The system stores owner ratings in the Rates table. The current schema already uses a check expression for ratings between 0 and 10, and this is a meaningful domain-level rule for the veterinarian evaluation feature.

This constraint prevents invalid veterinarian scores from being stored and ensures consistency in rating analysis.

```
ALTER TABLE Rates
ADD CONSTRAINT chk_rating_range
CHECK (rating BETWEEN 0 AND 10);
```

5. Implementation Plan

5.1 Frontend Development: Next.js with TypeScript

Next.js with TypeScript will be used to develop the frontend of the Veterinary Clinic Chain Management System. It will provide the user interface for different roles such as pet owners, veterinarians, and managers/administrators. The frontend will allow users to perform operations such as booking appointments, viewing medical records, managing inventory, and processing billing information through a web-based interface. The frontend will communicate with the backend by sending HTTP requests to API endpoints implemented on the server.

5.2 Backend Development: Flask

Flask will be used to implement the backend of the system. Flask will handle server-side logic such as authentication, appointment scheduling, medical record management, inventory tracking, and billing operations. The backend will process requests received from the frontend and execute the required raw SQL queries to interact with the database.

5.3 Database Management: PostgreSQL

PostgreSQL will be used as the main database management system of the Veterinary Clinic Chain Management System. It will store all relational data, including users, pets, veterinarians, appointments, prescriptions, inventory items, and billing records. The database will support a multi-branch clinic structure where each branch manages its own appointments and inventory while maintaining consistent patient records across the entire clinic chain. All database operations will be performed using raw SQL queries, in accordance with the project requirements. 3.4 Integration The system will follow a client-server architecture in which the frontend communicates with

the backend through HTTP requests. The backend will execute SQL queries on the PostgreSQL database and return the results to the frontend. All database interactions will occur on the server side to ensure data security, reliability, and proper system operation.